AI-Based Facial Emotion Analysis System Using Deep Learning

Exam Roll : 1092
Reg: 578
Session: 2020–2021

Exam Roll : 1087
Reg: 591
Session: 2020–2021

A Project Submitted in Partial Fulfillment of the Requirements for the Degree of Bachelor of Science in **Computer Science and Engineering (CSE)**

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Shyamoli Engineering College
Dhaka 1207, Bangladesh

August 2026



Shyamoli Engineering College
Dhaka 1207, Bangladesh

AI-Based Facial Emotion Analysis System Using Deep Learning

Exam Roll : 1092
Reg: 578
Session: 2020–2021

Exam Roll : 1087
Reg: 591
Session: 2020–2021

Submitted to the Department of Computer Science and Engineering in Partial Fulfillment of the Requirements for the Degree of Bachelor of Science in Computer Science and Engineering

Evaluation Committee

Teacher 1: _____

Teacher 2: _____

Teacher 3: _____

Declaration

We hereby declare that the project report entitled “AI-Based Facial Emotion Analysis System Using Deep Learning” is an authentic record of our own work carried out under the supervision of our respected supervisor in the Department of Computer Science and Engineering, Shyamoli Engineering College, as a partial fulfillment of the requirements for the degree of Bachelor of Science (B.Sc.) in Computer Science and Engineering for the academic session 2020–2021. We further declare that this project work is original and has been completed by us. All sources of information used in this project have been properly acknowledged and referenced. This report has not been submitted, either in whole or in part, to any other institution or university for the award of any degree or diploma. We accept full responsibility for the contents of this report and confirm that the work presented here reflects our own effort, research, and implementation.

Authors

Exam Roll : 1092
Reg: 578
Session: 2020–2021

&

Exam Roll : 1087
Reg: 591
Session: 2020–2021

Department of Computer Science and Engineering
Shyamoli Engineering College

Approval

This is to certify that the project entitled “AI-Based Facial Emotion Analysis System Using Deep Learning” has been successfully carried out by the students of the Department of Computer Science and Engineering, Shyamoli Engineering College, under my direct supervision. This project has been completed as a partial fulfillment of the requirements for the degree of Bachelor of Science (B.Sc.) in Computer Science and Engineering for the academic session 2020–2021. The work embodied in this project is original and has been carried out by the students themselves under my guidance. To the best of my knowledge, this project has not been submitted elsewhere for the award of any other degree or diploma. The students demonstrated sincerity, dedication, and technical competence during the development of this project. Their efforts in research, design, implementation, and documentation are highly appreciated. I wish them every success in their future academic and professional endeavors, and I approve this report for submission.

Supervisor Approval

_____

Departmental Approval

_____

Head, Department of Computer Science and Engineering

Shyamoli Engineering College

Abstract

Facial Emotion Recognition (FER) is a field that brings together artificial intelligence, computer vision, and psychology to identify human emotions from facial expressions. In this project, a deep learning system was developed using a Convolutional Neural Network (CNN) to classify emotions based on facial images. The model was trained on the FER2013 dataset and was set up to recognize five emotion categories: Happy, Sad, Angry, Surprise, and Neutral. Image preprocessing techniques such as resizing, normalization, and data augmentation were used to improve the model's performance and ability to generalize. The CNN architecture extracts key facial features through convolutional and pooling layers, then uses fully connected layers for classification. To improve accuracy, hyperparameters were tuned, including the learning rate, batch size, and number of epochs, and regularization techniques such as dropout were applied to reduce overfitting. The model achieved good accuracy and showed consistent performance across different testing scenarios.

For real-world use, the system was deployed with Streamlit. This setup allows for real-time emotion detection through live webcam input and image uploads, with an interactive interface that helps users easily analyze facial expressions. The complete implementation — including the development environment, the CNN, the database layer, and the deployed web application — was further verified through a structured functional testing process covering unit, integration, system, and user-acceptance levels before being published for public use. This project showcases how deep learning can be used in practical settings, such as healthcare, education, and human-computer interaction.

Acknowledgement

I would like to thank my supervisor for his ongoing support, expert guidance, and helpful advice throughout this project. His feedback, patience, and mentorship have greatly influenced the direction and success of this research. His encouragement pushed me to overcome challenges and finish this project. I also want to thank the respected faculty members of the Department of Computer Science and Engineering at Shyamoli Engineering College for their support, valuable suggestions, and academic help during my studies. Their teachings provided a solid foundation for this project.

I acknowledge the developers and research communities who created and maintained the open-source tools and technologies used in this project, including Python, TensorFlow, Keras, OpenCV, Scikit-learn, and Streamlit. Their work made the development and implementation of this system much easier. I am especially grateful to my classmates and friends for their cooperation, thoughtful discussions, and technical support during the project. Their suggestions and help allowed me to solve many technical issues. Finally, I want to thank my beloved parents and family for their unconditional love, encouragement, and support. Their patience and motivation gave me the strength and confidence to complete this project successfully.

Table of Contents

Evaluation Committee ii
Declaration iii
Approval iv
Abstract v
Acknowledgement vi
Table of Contents vii
List of Figures xi
List of Tables xii
List of Equations xiii
List of Abbreviations and Symbols xiv

Chapter 1: Introduction

1.1 Introduction 1
1.2 Motivation 2
1.3 Problem Statement 3
1.4 Objectives 4
1.5 Research Questions 4
1.6 Significance of the Study 5
1.7 Scope and Limitations 5
1.8 System Overview 6
1.9 Organization of the Report 7

Chapter 2: Background Study

2.1 Related Works 9
2.2 Models of Human Emotion 9
2.3 Classical, Handcrafted-Feature Approaches to FER 10
2.4 Deep Learning–Based Approaches to FER 12
2.5 Facial Emotion Recognition Datasets 13
2.6 Theoretical Background of Convolutional Neural Networks 14
2.7 Evaluation Metrics: Theoretical Background 16
2.8 Existing Systems 17
2.9 Bias, Fairness, and Ethical Considerations in FER 18
2.10 Research Gap 19
2.11 Summary Table of Positioning 20
2.12 Summary 20

Chapter 3: System Design

3.1 System Architecture 21
3.2 Design Goals and Non-Functional Requirements 21
3.3 Data Flow Diagram (Level 0) 22
3.4 Data Flow Diagram (Level 1) 22
3.5 Use Case Diagram 23
3.6 Sequence of Operations 24
3.7 Entity–Relationship (ER) Diagram 25
3.8 System Modules 25
3.9 API and Interface Contracts Between Modules 26
3.10 Interface Design Considerations 27
3.11 Hardware and Software Requirements 27
3.12 Deployment Environment Specification 28
3.13 Design Constraints and Assumptions 28
3.14 Summary 29

Chapter 4: Methodology

4.1 Dataset Description (FER2013) 30
4.2 Exploratory Data Analysis 31
4.3 Data Preprocessing 31
4.4 Data Augmentation Strategy 32
4.5 Model Building (Custom CNN) 32
4.6 Hyperparameter Selection 33
4.7 Training and Validation 34
4.8 Model Selection and Checkpointing 34
4.9 Tools and Technologies 35
4.10 Evaluation Methodology 35
4.11 Reproducibility 35
4.12 Summary 36

Chapter 5: Implementation

5.1 Introduction 37
5.2 Environment Setup 37
5.3 Tools and Technologies Used 38
5.4 Mapping the Architecture to Code 39
5.5 Step-by-Step Development Process 40
5.6 Model Implementation (Custom CNN) 41
5.7 Data Pipeline Implementation 42
5.8 Database Integration 42
5.9 Error Handling and Input Validation 43
5.10 Streamlit Application Development and Deployment 43

5.11 Performance Optimization 44
5.12 Version Control and Project Structure 44
5.13 Logging and Monitoring 45
5.14 Configuration Management 45
5.15 Accessibility Considerations 45
5.16 Interface Snapshots 46
5.17 Key Code Snippets 48
5.18 Challenges Faced During Implementation 49
5.19 Summary 50

Chapter 6: Testing

6.1 Introduction 51
6.2 Testing Objectives 51
6.3 Test Strategy 51
6.4 Levels of Testing 52
6.5 Test Environment 52
6.6 Test Case Design 53
6.7 Unit Testing Detail 54
6.8 Integration Testing Detail 54
6.9 Testing Tools and Techniques 55
6.10 Test Execution Summary 55
6.11 Requirements Traceability Matrix 56
6.12 Usability Testing Results 56
6.13 Regression Testing 57
6.14 System Output (Deployed Application) 57
6.15 Defect Log 59
6.16 Summary 60

Chapter 7: Results and Evaluation

7.1 Introduction 61
7.2 Performance Metrics 62
7.3 Training Curve Analysis 62
7.4 Confusion Matrix Analysis 63
7.5 Per-Class Analysis 64
7.6 Comparison with Human-Level Performance 64
7.7 System Evaluation 65
7.8 Computational Efficiency Analysis 65
7.9 Comparative Analysis 66
7.10 Ablation Study 66
7.11 Reliability of the Reported Results 67
7.12 Threats to Validity 67

7.13 Discussion 68
7.14 Revisiting the Research Questions 69
7.15 Summary 70

Chapter 8: Conclusion and Future Work

8.1 Introduction 71
8.2 Summary of Findings 71
8.3 Achievement of Objectives 72
8.4 Challenges Encountered 72
8.5 Lessons Learned 73
8.6 Future Enhancements 73
8.7 Recommendations for Future Researchers 74
8.8 Ethical Considerations and Responsible Use 74
8.9 Sustainability and Maintenance 75
8.10 Conclusion 76

References 77

Appendix A: Installation and User Guide

A.1 Purpose of This Appendix 80
A.2 Prerequisites 80
A.3 Local Installation Steps 80
A.4 Using the Web Application 81
A.5 Troubleshooting 82

Appendix B: Extended Source Code Listing

B.1 Purpose of This Appendix 83
B.2 Preprocessing Module with Input Validation 83
B.3 Database Access Module 84
B.4 Representative Unit Test 85
B.5 Notes on Reproducing These Listings 86

Appendix C: Glossary of Terms 87

Appendix D: Sample Testing Log Excerpt 89

Appendix E: Data and Model Card 90

List of Figures

Figure 1.1: System Overview of Face Emotion Detection Using Deep Learning 7
Figure 2.1: CNN Architecture for Emotion Detection 16
Figure 2.2: Comparison of Existing Emotion Detection Systems 17
Figure 2.3: Data Preprocessing Pipeline for Facial Emotion Detection 18
Figure 3.1: System Architecture Diagram 21
Figure 3.2: Data Flow Diagram – Level 0 22
Figure 3.3: Data Flow Diagram – Level 1 23
Figure 3.4: Use Case Diagram 23
Figure 3.5: ER Diagram for Stored Results 25
Figure 4.1: Training/Validation Workflow and Deployment Flow with Streamlit 34
Figure 5.1: Model Training Workflow 38
Figure 5.2: Streamlit Application Layout 41
Figure 5.3: Database Schema Diagram 42
Figure 5.4: Streamlit App – Home Interface (Upload & Webcam options) 46
Figure 5.5: Start Webcam Detection – Live Capture View 46
Figure 5.6: Real-time Emotion Detection Result 47
Figure 5.7: Recent Emotions and Feedback Messages 47
Figure 6.1: Deployed application — home screen at emotion-rayhan.streamlit.app, showing the Webcam Snapshot, Upload Image Detection, and Live Emotion Analysis entry points 57
Figure 6.2: Deployed application — real-time detection output with predicted emotion and confidence 58
Figure 6.3: Deployed application — Live Analytics view: emotion-count bar chart and confidence-over-time line chart 59
Figure 7.1a: Training and Validation Accuracy over Epochs 62
Figure 7.1b: Training and Validation Loss over Epochs 63
Figure 7.2: Confusion Matrix for Emotion Detection Model 63
Figure 7.3: Accuracy Comparison – Proposed Model vs Existing Methods 66

List of Tables

Table 2.1: Summary of selected studies and datasets commonly used for facial emotion detection 13

Table 2.2: Comparison of commonly used facial emotion recognition datasets 13

Table 2.3: Positioning of this project's approach against the categories of approach surveyed in this chapter 20

Table 3.1: Use case descriptions 24

Table 3.2: Function-level contracts between adjacent modules 26

Table 3.3: Deployment environment specification (Streamlit Community Cloud) 28

Table 4.1: Approximate class distribution of the FER2013 dataset, full seven-class set and the five-class subset used in this project 30

Table 4.2: Final hyperparameter values and the range considered during tuning 34

Table 5.1: Tools and technologies used in implementation 39

Table 5.2: Architecture modules mapped to implementation artifacts 39

Table 6.1: Functional test cases 53

Table 6.2: Test execution summary by module 55

Table 6.4: Requirements traceability matrix: non-functional requirements to verifying test cases 56

Table 6.3: Defect log 60

Table 7.1: Performance metrics for each emotion class 62

Table 7.2: Comparison between offline and real-time detection performance 65

Table 7.3: Computational footprint of the proposed CNN compared with deeper reference architectures 65

Table 7.4: Ablation study: validation accuracy under architectural variations, all else held constant 67

Table A.1: Common installation and usage issues and their resolutions 82

Table C.1: Glossary of technical terms used in this report 87

Table D.1: Excerpt from the manual system-testing log 89

Table E.1: Dataset card 90

Table E.2: Model card 90

List of Equations

Equation 2.1: Convolution operation 14
Equation 2.2: ReLU activation 14
Equation 2.3: Max pooling 15
Equation 2.4: Softmax function 15
Equation 2.5: Categorical cross-entropy loss 15
Equation 2.6: Accuracy 16
Equation 2.7: Precision 16
Equation 2.8: Recall 16
Equation 2.9: F1-score 16

List of Abbreviations

Abbreviation	Description
CNN	Convolutional Neural Network
FER2013	Facial Expression Recognition Dataset 2013
ReLU	Rectified Linear Unit
DFD	Data Flow Diagram
ER	Entity Relationship
UI	User Interface
DB	Database
GPU	Graphics Processing Unit
HCI	Human-Computer Interaction
API	Application Programming Interface
UAT	User Acceptance Testing

List of Symbols

Symbol	Description
η	Learning rate
α	Regularization parameter
W	Model weights
b	Bias value
σ	Activation function
L	Loss function

Chapter 1: Introduction

1.1 Introduction

Emotion detection from facial expressions has become an important area of research in artificial intelligence. It aims to connect human emotions with machine understanding. Thanks to improvements in deep learning and computer vision, modern systems can now interpret emotional cues accurately. This technology is significant in various fields such as virtual reality, robotics, e-learning, and mental health assessment. Facial expressions are a universal form of communication, making them an effective way to recognize human emotions (22, 23). The main goal of this project is to develop a deep learning-based model that can classify human emotions from facial expressions. To achieve this, a Convolutional Neural Network (CNN) is used because of its strong performance in image classification tasks (7, 8).

Human beings communicate a great deal of information without ever speaking a word. A raised eyebrow, a tightened jaw, a fleeting smile — each of these carries meaning that a purely text- or speech-based system would miss entirely. As computing systems become more deeply embedded in everyday life, from customer-service kiosks to classroom software to in-car assistants, the absence of this channel of communication becomes a genuine limitation. A system that can read a face, even imperfectly, is a system that can respond to a person rather than merely to their explicit input. This project is an attempt to build one small, well-scoped piece of that larger capability: a model that looks at a face and produces a reasonable guess at the emotion it expresses, wrapped in an application simple enough that anyone can try it without installing anything or writing a line of code.

The model is trained using the FER2013 dataset, which contains labeled grayscale images of different emotional states (1). The system classifies emotions into categories like Happy, Sad, Angry, Surprise, and Neutral. Before training, the input images undergo preprocessing, including resizing, normalization, and data augmentation techniques to improve model accuracy and generalization. The CNN architecture automatically extracts key facial features using convolutional and pooling layers, followed by fully connected layers for the final classification. To improve performance, various hyperparameters such as learning rate, batch size, and number of epochs are optimized. Regularization techniques, including dropout, are also used to reduce overfitting and enhance model stability.

For practical implementation, the trained model is integrated into a web-based application using the Streamlit framework (4). This interface allows users to interact with the system by uploading images or capturing real-time input through a webcam. The system processes the input and gives instant emotion predictions, making it suitable for real-time applications. Overall, this project shows how deep learning can effectively recognize emotions and provides a base for developing intelligent systems that can understand human emotions. It can also be extended for use in healthcare, education, and human-computer interaction.

1.2 Motivation

The motivation behind this project comes from the growing demand for emotionally intelligent systems in today's connected world. As artificial intelligence evolves and becomes a part of daily life, people expect machines to perform tasks efficiently and understand human emotions in a meaningful way. Traditional systems mainly focus on logical processing and data-driven decision making, often ignoring the emotional context of human interactions. However, human communication is naturally emotional, and adding emotional awareness to intelligent systems can greatly improve the quality of interactions between people and machines.

Emotion-aware systems can change how users interact with technology by making these exchanges more natural, adaptable, and personalized. When a system can recognize a user's emotional state, it can adjust its responses, leading to a more engaging and human-like experience. For instance, an application that detects frustration can offer extra support or simplify its interface, while a system that picks up on positive emotions can encourage user engagement. This ability to respond in real time makes emotion detection a key part of developing next-generation intelligent systems.

The value of this technology shows in its many applications across different fields. In education, emotion detection can help create adaptive learning environments where educational content is adjusted based on students' emotional responses. If a student seems confused or disengaged, the system can modify the content, provide extra explanations, or change the difficulty level to improve understanding and keep interest. This can lead to better learning outcomes and a more personalized educational experience.

In healthcare, emotion recognition systems can be crucial for monitoring mental health and providing support. By analyzing facial expressions, these systems can identify early signs of stress, anxiety, or other emotional states worth attention. Early detection allows for timely intervention, which can improve outcomes and assist healthcare professionals in understanding a patient's presentation. Additionally, such systems can be integrated into telemedicine platforms, enabling remote emotional assessments and support. In customer service, emotion-aware AI systems can greatly enhance the quality of interactions between businesses and their customers. By analyzing emotional cues, businesses can better understand customer needs and respond more effectively.

There is also a personal, practical motivation behind the choice of tools and scope for this project. A system that depends on a paid cloud API, a constant internet connection, or specialized hardware is of little use to a student, a small classroom, or a clinic with limited resources. From the outset, this project was framed as a lightweight, self-contained system: trainable on a single consumer GPU or even a capable CPU, deployable as a single web application, and free to run indefinitely once built. That constraint shaped nearly every later decision — from the choice of a compact custom CNN over a heavyweight pretrained network, to the choice of Streamlit for deployment, to the choice of SQLite over a managed database service.

1.3 Problem Statement

Even though artificial intelligence and computer vision have advanced rapidly, understanding human emotion through facial images poses many complex difficulties. Emotions are often very nuanced and are affected by many different contextual factors; therefore, they are often difficult even for humans to read accurately. When machines utilize facial images for automated facial emotion recognition, these factors complicate the recognition process further because the machine cannot assess the face without visual data and patterns based on learned information. Therefore, designing a facial emotion recognition system that is dependable and accurate requires addressing a number of different technical and environmental issues.

One major challenge associated with recognizing facial expressions is variation in lighting conditions. Change in lighting has a dramatic impact on how a person looks, which makes the input of the facial features relatively inconsistent. In shaded, overexposed, or low-illumination conditions, key regions such as the eyes, mouth, and contours of the face can be obscured; consequently, it becomes difficult for the model to extract consistent evidence on which to base its prediction.

Another important factor is head-pose variation. Capturing a face from different angles can easily change the way the facial expression is interpreted. The majority of current datasets consist of frontal views, when in reality, face recognition applications can include views of a subject from both side profile and tilt angles. Facial occlusions also create significant problems for establishing an emotion. Common occlusions include glasses, masks, hands, and hair — all of which can cover key areas of a face that are necessary for establishing emotional context, such as the eyes and mouth. When these areas are occluded, an emotion detection model will have more difficulty classifying the expression correctly. Other environmental factors, such as background clutter and camera quality, also create challenges for real-time emotion detection.

A further, less visible difficulty is the labeling problem itself. Unlike, say, classifying a photograph as containing a cat or a dog, classifying an emotion asks annotators to agree on a subjective, culturally influenced judgment from a still image with no audio, no body language, and no context. Two careful human annotators can reasonably disagree on whether a given face looks 'sad' or merely 'neutral', and this disagreement shows up directly as label noise in any dataset built from crowd-sourced or web-scraped images, FER2013 included. A model trained on such labels inherits some of that ambiguity, and no amount of architectural sophistication can fully undo it — a fact that shapes how the results in Chapter 7 should be read, and why the confusion patterns between adjacent emotions (Sad/Neutral, Angry/Sad) are treated as expected behaviour rather than simple errors.

Finally, there is a deployment-level problem that sits alongside the modelling problem: even a highly accurate offline model is only useful if it can be reached easily by the people who need it. Many published FER research systems stop at a trained model and a reported accuracy figure, with no accompanying interface a non-technical user could actually run. This project treats that gap as part of the problem statement, not as an afterthought — the

system is only considered complete once it is reachable through an ordinary web browser, on both desktop and mobile, without any setup on the user's side.

1.4 Objectives

The goal of this project is to develop a Convolutional Neural Network (CNN)–based emotion detection system using the FER2013 facial expression dataset to classify human emotions from facial images, focusing on emotions such as happiness, sadness, anger, surprise, and neutrality. To improve the model's performance and classification accuracy, several data preprocessing techniques are applied, including image normalization, resizing, and grayscale processing. In addition, data augmentation methods such as rotation, flipping, zooming, and shifting are used to increase the diversity of the training data and reduce overfitting. After training the CNN model, its effectiveness is evaluated using standard performance metrics such as accuracy, precision, recall, and F1-score (6). Finally, the trained model is integrated into a real-time emotion recognition interface using Streamlit (4), allowing users to upload images or use a webcam to detect facial emotions, demonstrating the practical application and real-world usability of the system.

The specific objectives pursued in this project can be grouped as follows.

Primary Objectives:

1. Design and train a custom CNN architecture capable of classifying facial images into five emotion categories with accuracy competitive with published FER2013 benchmarks.
2. Build a complete, reproducible preprocessing pipeline (face detection, grayscale conversion, resizing, normalization, and augmentation) that can be applied identically at training and at inference time.
3. Deploy the trained model behind an interactive, browser-based interface that supports both static image upload and live webcam capture.

Secondary Objectives:

1. Provide a lightweight persistence layer that logs predictions for later review, without requiring a dedicated database server.
2. Verify the system's software behaviour, not just its model accuracy, through a structured testing programme covering unit, integration, system, and user-acceptance levels.
3. Document the design, implementation, and evaluation thoroughly enough that another developer could reproduce or extend the system from this report alone.

1.5 Research Questions

To keep the project's scope disciplined and its evaluation meaningful, the work reported here was organized around four concrete research questions rather than the single broad goal of 'building an emotion detector'. Each question maps directly onto a later chapter of this report, and the answers arrived at are summarized in Chapter 8.

1. RQ1 — Architecture: Can a small, custom-designed CNN, trained from scratch rather than fine-tuned from a large pretrained network, reach an accuracy on FER2013 that is competitive with the literature while remaining light enough for real-time, CPU-only inference?
2. RQ2 — Preprocessing: How much of the achievable accuracy on a noisy, low-resolution dataset like FER2013 can be recovered through preprocessing and augmentation choices alone, without changing the underlying model?
3. RQ3 — Deployment: What does it take, beyond a saved model file, to turn a trained classifier into an application that behaves predictably for real users on real devices — including unusual, low-quality, or invalid input?
4. RQ4 — Verification: Can the correctness of the surrounding software (not just the accuracy of the model) be established through a structured, repeatable testing process, in the same way it would be for any other web application?

1.6 Significance of the Study

This project sits at the intersection of three separate but connected concerns: model design, software engineering, and accessibility of the finished tool. Much of the published research on FER2013 stops once a benchmark accuracy number has been reported; comparatively little attention is given to what it takes to turn that model into something a non-specialist can actually use. By treating the deployed, publicly reachable Streamlit application as the true deliverable — rather than treating the trained model file as the end point — this project tries to close some of that gap for a specific, well-scoped use case.

The significance of the work can be summarized along three lines. First, from an academic standpoint, it demonstrates that a lightweight, from-scratch CNN can reach competitive accuracy on FER2013 without relying on transfer learning from a large pretrained backbone, which matters for settings where compute budgets are small. Second, from a software engineering standpoint, it demonstrates a full pipeline — from raw dataset to tested, deployed web application — including the testing discipline (Chapter 6) that is often missing from student machine-learning projects. Third, from a practical standpoint, the resulting application is genuinely usable today by anyone with a browser, at zero marginal cost, which is a meaningful contribution in resource-constrained educational and small-clinic settings where a subscription-based commercial emotion-recognition API is not a realistic option.

1.7 Scope and Limitations

This project focuses on developing an emotion recognition system that identifies five different facial emotions: Happy, Sad, Angry, Surprise, and Neutral (neither happy nor sad). The system detects these emotions by analyzing facial images only, without incorporating other multimodal emotion detection methods such as voice recognition, body gestures, or physiological signals. The model is trained using the FER2013 dataset, where all facial images are 48×48 pixels in size and converted to grayscale, allowing the Convolutional Neural Network (CNN) to focus on important facial features rather than

colour information. The system is designed to analyze facial images and predict the corresponding emotion, and it is integrated into a real-time interface using Streamlit, enabling users to test the model through uploaded images or webcam input.

However, this project has several limitations. The accuracy of the model depends on the quality and diversity of the dataset, which may contain biases related to lighting conditions, facial orientation, and demographic representation. Since the images are low-resolution grayscale images, some subtle facial details may be lost, which may affect prediction accuracy. In addition, the model may perform less effectively when faces are partially covered, poorly illuminated, or captured from unusual angles. The real-time performance of the system may also depend on hardware capabilities, such as processing power and camera quality. Furthermore, the system is limited to recognizing only the five predefined emotions, and it may not accurately detect complex or mixed emotional expressions.

It is also worth being explicit about what this project does not attempt. It does not attempt continuous emotion tracking across a video sequence with temporal smoothing; each frame or uploaded image is classified independently. It does not attempt to infer intensity of emotion (e.g. mild versus intense happiness) — only the most likely discrete category. It does not attempt cross-cultural validation of the emotion categories themselves, and it relies on the categorical model of emotion (Ekman's basic emotions) (22, 23) rather than a dimensional model such as valence-arousal. These are reasonable directions for future work (Section 8.4), but they are explicitly out of scope for the system evaluated in this report.

1.8 System Overview

As presented in Figure 1.1, the entire face expression detection system can be explained in an end-to-end manner. The first step in this process is to acquire an image, either by uploading it from a PC or capturing it with a web camera. The acquired image is pre-processed by being resized, normalised, and converted to a greyscale image prior to passing through the CNN model. Once processed through the CNN model, the network extracts features through the multiple convolutional layers and then classifies what emotion the facial expression shows.

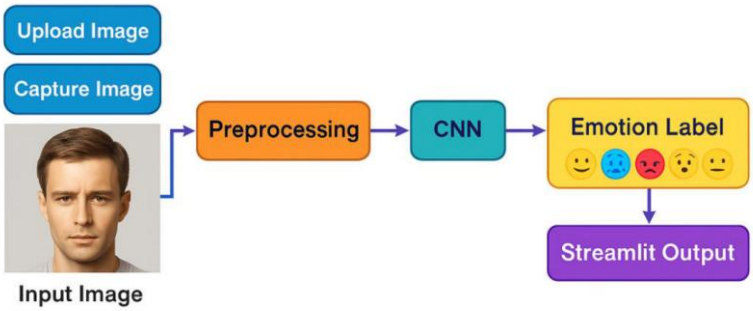


Figure 1.1: System Overview of Face Emotion Detection Using Deep Learning

As seen in Figure 1.1, the system comprises five main modules: input acquisition via camera or file upload, preprocessing for noise reduction and normalization, CNN-based feature extraction, emotion classification, and a Streamlit output interface for displaying predictions in real time.

1.9 Organization of the Report

The remainder of this report is organized into seven further chapters, an appendix, and a list of references, structured to move from background and theory through to a working, verified system.

- Chapter 2 (Background Study) surveys prior work in facial emotion recognition, from classical handcrafted-feature methods through modern CNN-based approaches, and lays out the theoretical foundations — convolution, activation, pooling, and loss — that the model in later chapters builds on.
- Chapter 3 (System Design) presents the system architecture, data flow diagrams, use case diagram, entity-relationship model, and the hardware/software requirements for the system.
- Chapter 4 (Methodology) describes the dataset, the preprocessing pipeline, the CNN architecture, and the training and validation procedure used to produce the final model.
- Chapter 5 (Implementation) documents how that methodology was actually built — the development environment, the tools used, a step-by-step account of the build

process, and the key code behind the model, the database, and the deployed application.

- Chapter 6 (Testing) describes how the implementation was verified through unit, integration, system, and user-acceptance testing, and reports the test cases and their outcomes.
- Chapter 7 (Results and Evaluation) reports the trained model's quantitative performance — accuracy, per-class precision/recall/F1-score, the confusion matrix — and discusses what that performance means in practice.
- Chapter 8 (Conclusion and Future Work) summarizes what was achieved against the objectives of Section 1.4 and the research questions of Section 1.5, and outlines directions for future work.
- The Appendices provide a self-contained installation and user guide for the deployed application, together with an extended source-code listing for readers who want more implementation detail than Chapter 5 includes inline.

Chapter 2: Background Study

2.1 Related Works

Recognition of emotions in the face (FER) is a rapidly growing and active area of research in fields such as artificial intelligence and computer vision (13). It is critical for intelligent systems to be able to accurately recognize and interpret human emotions through facial expressions in order to provide for natural human interactions. The ability to recognize emotions improves not only the way human beings interact with computers, but also has large implications for education, healthcare, marketing, security, and mental health monitoring. Therefore, FER has gained much interest from both academic and industrial researchers, with new methodologies and technologies continuously being developed.

The remainder of this chapter is organized to move from the psychological question of what an emotion even is (Section 2.2), through the two broad eras of computational approaches to recognizing it — handcrafted features (Section 2.3) and deep learning (Section 2.4) — to the datasets those approaches are trained and benchmarked on (Section 2.5), the mathematical machinery a CNN relies on (Section 2.6), the metrics used to judge a trained classifier (Section 2.7), the commercial and open-source systems already available (Section 2.8), and finally the specific gap this project sets out to address (Section 2.10).

2.2 Models of Human Emotion

Before a machine can be asked to recognize an emotion, there has to be some agreed-upon set of categories or dimensions to recognize it into, and psychology offers more than one answer to that question. The dominant model in FER research, and the one adopted in this project, is Paul Ekman's theory of basic emotions (22, 23). Ekman's cross-cultural studies argued that a small set of emotions — commonly listed as happiness, sadness, anger, fear, surprise, disgust, and a neutral resting state — are expressed and recognized in essentially the same facial configuration across widely different cultures, which suggested a biological rather than purely learned basis for facial expression. This categorical view is convenient computationally because it turns emotion recognition into an ordinary multi-class classification problem, which is exactly how FER2013 and most other public datasets are labeled, and exactly how the model in this project is trained (Chapter 4).

Ekman's own later work, the Facial Action Coding System (FACS) (22), takes the categorical idea one level deeper: rather than labeling a face directly with an emotion word, FACS decomposes an expression into a set of Action Units (AUs), each corresponding to the contraction of a specific facial muscle or muscle group — for example, AU12 (lip corner puller) is strongly associated with a smile. An emotion label can then, in principle, be derived from a combination of active Action Units. FACS is influential in the FER literature because it gives a physically grounded, annotator-independent vocabulary for describing an expression, and several datasets (including the CK+ dataset described in Section 2.5) provide AU annotations alongside emotion labels. This project does not use AU-level annotations directly — the FER2013 dataset does not provide them — but FACS is worth understanding because it explains why certain emotions are systematically harder

to tell apart than others: Sad and Angry share several activated brow and mouth Action Units, which is a large part of why they are the most frequently confused pair in the results reported in Chapter 7.

An alternative to the categorical view is the dimensional model of emotion, most commonly represented as Russell's circumplex model, which places any emotional state as a point in a continuous two-dimensional space defined by valence (how positive or negative the feeling is) and arousal (how calm or excited it is). Modern large-scale datasets such as AffectNet (12) provide both categorical labels and continuous valence-arousal annotations, reflecting a growing interest in this richer representation. The dimensional model has the advantage of being able to represent intensity and blended emotional states that do not map cleanly onto a single discrete category, but it is harder to annotate consistently and harder to evaluate with simple classification metrics. Given the scope defined in Section 1.7, this project deliberately stays with the categorical model — it is simpler to annotate, simpler to evaluate, and matches the dataset actually used for training.

2.3 Classical, Handcrafted-Feature Approaches to FER

The development of FER systems in the early days used mainly manually constructed features to extract the characteristics of a person's face. Common techniques for extracting features from face images included Gabor filters, Local Binary Patterns (LBP) (24), and Eigenfaces. Gabor filters are used to derive texture and edge information of the skin at localized regions of a face; a bank of Gabor filters at different orientations and spatial frequencies is convolved with the image, and the resulting filter responses form the feature vector passed to a classifier such as a Support Vector Machine (SVM) (28). The goal of LBP is to use pixel-intensity patterns to extract the structure of facial features: each pixel is compared against its immediate neighbours, and the pattern of 'brighter' or 'darker' comparisons is encoded as a small binary number, producing a texture descriptor that is notably robust to monotonic changes in lighting. Eigenfaces use principal component analysis to derive a low-dimensional representation of an image, projecting each face onto a small set of the most significant variance-capturing basis images ('eigenfaces') learned from a training set.

A related family of handcrafted descriptors, Histogram of Oriented Gradients (HOG), computes local gradient-orientation histograms over a grid of image cells and was widely used both for face detection and as an input representation for emotion classifiers before the deep learning era. Once a handcrafted feature vector had been produced by one of these methods, a separate, shallow classifier — most commonly an SVM (28), though boosted classifiers and k-nearest-neighbour approaches also appear in the literature — was trained on top of it to map the feature vector onto an emotion label. Feature extraction and classification were, in other words, two independent stages designed and tuned separately by the researcher.

Although these techniques effectively captured common variations of faces and provided acceptable recognition rates in controlled environments, they had significant limitations due to extreme sensitivity to environmental conditions, including light levels, head

orientation, and occlusion; extensive manual tuning of the feature-extracting methods also limited their adaptability and scalability to today's real-world applications. Critically, because the feature-extraction stage was fixed in advance and not learned from data, these pipelines could not automatically discover which visual patterns were actually most discriminative for a given dataset — a limitation that the deep learning approaches discussed next were specifically designed to remove.

2.4 Deep Learning–Based Approaches to FER

The introduction of deep learning has brought about a significant change to FER studies, providing an efficient, automatic solution to the task of classifying and extracting features (7, 10). Landmark deep architectures such as VGGNet (17) and GoogLeNet (18) demonstrated that increasing network depth, together with techniques like batch normalization (19) and dropout (20) to keep such deeper networks trainable, could substantially improve image-classification accuracy — a lesson subsequently carried into FER research. CNNs have particularly changed the way FER is approached, allowing for automatic learning of the hierarchical structure of a feature from raw pixel data. By allowing the automatic extraction of features from images without the need for manually designed or handcrafted features, CNNs enable the extraction of both low-level (edge and texture) and high-level (identity and emotion-related) descriptive features from images (8, 9).

Within CNN-based FER, the literature can be loosely grouped into three generations. The first generation, exemplified by the early work of Goodfellow et al. (1) that introduced FER2013 itself, applied comparatively shallow CNNs — a handful of convolutional and pooling layers — trained directly on 48×48 grayscale crops, and reported accuracies in the high seventies on the FER2013 test set. The second generation adapted deeper architectures originally designed for general-purpose image classification, most notably VGG-Face variants and residual networks such as ResNet50 (9), sometimes via transfer learning from weights pretrained on large face-recognition or ImageNet-style datasets (8). These deeper networks generally improved accuracy on larger, more diverse datasets such as AffectNet (12) at the cost of a substantially larger parameter count and slower inference, and it is exactly this trade-off — a small accuracy gain against a large compute cost — that motivates the lightweight, custom architecture chosen for this project rather than a deeper transferred network (see Section 2.10 and the model design rationale in Section 4.3).

The third and most recent generation incorporates attention mechanisms and, increasingly, vision transformer blocks, allowing a network to learn to weight the eyes, brows, and mouth region more heavily than the rest of the face when producing a prediction, which more closely mirrors how a human FACS-trained annotator would read an expression. These architectures currently report the strongest published numbers on in-the-wild benchmarks like AffectNet and RAF-DB, but at the cost of even larger models and training sets than the second generation. Given the resource constraints and real-time, CPU-friendly deployment goal set out in Section 1.2, attention and transformer mechanisms are treated in this project as a direction for future work (Section 8.4) rather than a component of the current architecture.

Table 2.1: Summary of selected studies and datasets commonly used for facial emotion detection

Author/Year	Dataset Used	Model/Method	Accuracy (%)
Goodfellow et al. (2013)	FER2013	Deep CNN	79.2
Mollahosseini et al. (2017)	AffectNet	VGGFace CNN	66.8
Tang et al. (2013)	FER2013	CNN with ReLU	72.4
Zhao et al. (2020)	CK+	ResNet50 + Augmentation	89.5

These studies collectively highlight the growing dominance of CNN-based approaches in facial emotion recognition (1, 12). The evolution from datasets like FER2013 to more comprehensive datasets such as AffectNet (12) and CK+ (25) reflects significant improvements in data diversity, annotation quality, and real-world variability. These advancements, including deeper residual architectures (9), have enabled the development of more accurate and robust models capable of handling a wide range of facial expressions across different ages, ethnicities, and conditions, further strengthening the performance and generalization of modern emotion recognition systems. It is worth noting that accuracy figures across these studies are not perfectly comparable, since they are measured on different datasets, different train/test splits, and sometimes different subsets of emotion classes — a caveat that also applies to the comparison drawn later in Chapter 7 between this project's model and the figures quoted here.

2.5 Facial Emotion Recognition Datasets

The choice of training dataset has as much influence on a FER system's real-world behaviour as the choice of model architecture, because the dataset defines what the network is ever given the opportunity to learn. Table 2.2 summarizes the datasets most frequently referenced in the FER literature and in this report, alongside the dataset actually used for training in this project.

Table 2.2: Comparison of commonly used facial emotion recognition datasets

Dataset	Size	Emotion Labels	Collection Setting
FER2013 (1)	35,887 images (48×48, grayscale)	7 categories	Web-scraped, in-the-wild
CK+ (25)	593 sequences, 123 subjects	8 categories + Action Units	Lab-controlled, posed & spontaneous
JAFFE	213 images, 10 subjects	7 categories	Lab-controlled, posed
AffectNet (12)	~1,000,000 images (~420,000 annotated)	8 categories + valence/arousal	Web-scraped, in-the-wild
RAF-DB	~30,000 images	7 basic + 12 compound categories	Web-scraped, in-the-wild, crowd-annotated

FER2013 (1) was selected for this project for several converging reasons. It is large enough (35,887 labeled images) to train a CNN from scratch without immediately overfitting,

unlike the much smaller lab-controlled datasets such as CK+ or JAFFE. It is collected 'in the wild' from web images rather than staged in a laboratory, which better matches the uncontrolled webcam and upload conditions the deployed application actually faces (Section 3.1). Its images are already cropped, grayscale, and a fixed 48×48 size, which keeps the preprocessing pipeline and the resulting model small — directly supporting the real-time, CPU-friendly deployment goal from Section 1.2. And it is freely and permanently available without a data-usage agreement, unlike AffectNet, which requires a research-use request to the original authors. The trade-off, discussed honestly in Section 1.3 and revisited in Section 7.7, is that FER2013's low resolution and its labeling noise put a ceiling on achievable accuracy that a larger, higher-resolution dataset such as AffectNet would not.

2.6 Theoretical Background of Convolutional Neural Networks

Convolutional Neural Networks (CNNs) represent the foundation of modern image-based emotion recognition systems (10). Their layered structure enables the automatic learning of spatial hierarchies from raw pixel data. Each layer in a CNN plays a distinct role:

- Convolutional Layer – Extracts feature maps using learnable filters.
- Pooling Layer – Reduces spatial dimensions, minimizing computational cost.
- Activation Layer (ReLU) – Adds non-linearity for learning complex patterns.
- Flatten and Dense Layers – Transform features into classification outputs.
- Softmax Output – Converts logits into emotion probability distributions.

The convolution operation is expressed mathematically as:

$$f(i, j) = \Sigma_m \Sigma_n I(i + m, j + n) \times K(m, n) \quad \text{(Equation 2.1)}$$

where I denotes the input image and K represents the kernel (filter) used to extract spatial features from the image. Convolution helps the neural network detect patterns such as edges, textures, and facial structures that are important for emotion recognition. Each convolutional layer typically learns many such kernels in parallel, each producing its own feature map, so that the layer as a whole can respond to a variety of different local patterns at once. As successive convolutional layers are stacked, the effective receptive field — the region of the original input that a given output value is sensitive to — grows, which is what allows deeper layers to respond to progressively larger and more abstract structures, from a short edge segment in the first layer to something resembling an eye or a mouth corner several layers in.

After convolution, an activation function is applied to introduce non-linearity into the network. One commonly used activation function is the Rectified Linear Unit (ReLU), defined as:

$$ReLU(x) = max(0, x) \quad \text{(Equation 2.2)}$$

This function outputs the input value if it is positive; otherwise, it returns zero. ReLU helps the network learn complex patterns efficiently and speeds up the training process.

Compared with older activation functions such as the sigmoid or hyperbolic tangent, ReLU is cheap to compute and, crucially, does not saturate for large positive inputs, which reduces the vanishing-gradient problem that can otherwise stall learning in deeper networks.

To reduce the spatial dimensions of the feature maps and decrease computational complexity, a pooling operation is often applied. One common pooling technique is Max Pooling, which selects the maximum value from a specific region of the feature map. It can be expressed as:

$$P(i, j) = max(f(i + m, j + n)) \quad \text{(Equation 2.3)}$$

where (m, n) represents the coordinates within the pooling region. Beyond simply reducing the amount of computation needed in later layers, max pooling gives the network a small amount of translation invariance — a feature that shifts by a pixel or two between one image and the next still tends to produce the same pooled output — which is a useful property when faces in different photographs are never perfectly aligned.

In the final stage of the network, the extracted features are passed through fully connected layers to perform classification. The Softmax function, commonly used for multi-class classification, converts the output scores into probability values. It is defined as:

$$Softmax(z_i) = e^{(z_i)} / \Sigma_j e^{(z_j)} \quad \text{(Equation 2.4)}$$

where z_i represents the input score for class i, and the denominator is the sum of exponential values for all classes. The Softmax function ensures that the probabilities of all classes sum to 1, allowing the model to predict the most likely emotion category.

During training, the network's weights are adjusted to minimize a loss function that measures the difference between the predicted and true emotion labels. For multi-class classification, categorical cross-entropy is the standard choice and is defined as:

$$L = -\Sigma_i y_i \log(\hat{y}_i) \quad \text{(Equation 2.5)}$$

where y_i is the true (one-hot encoded) label for class i and $\hat{y}_i$ is the corresponding predicted probability produced by the Softmax layer. Minimizing L drives the predicted distribution closer to the true emotion label. In practice, this minimization is carried out iteratively by an optimization algorithm — this project uses Adam (21), described further in Section 4.4 — which uses the gradient of L with respect to every weight in the network, computed efficiently via backpropagation, to nudge each weight a small step in the direction that reduces the loss.

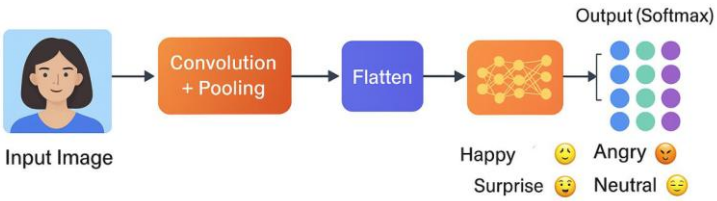


Figure 2.1: CNN Architecture for Emotion Detection

Figure 2.1: CNN Architecture for Emotion Detection

2.7 Evaluation Metrics: Theoretical Background

A trained classifier is only as useful as the tools available to judge it, and accuracy alone can be a misleading measure whenever a dataset's classes are not perfectly balanced — which, as Section 4.1 notes, FER2013 is not. This section defines the metrics used throughout Chapter 7, each expressed in terms of the four possible outcomes of a binary decision for a given class: true positives (TP), true negatives (TN), false positives (FP), and false negatives (FN).

$$Accuracy = (TP + TN) / (TP + TN + FP + FN)$$ (Equation 2.6)

Accuracy is the proportion of all predictions the model got right. It is intuitive and easy to report, but on an imbalanced dataset a model can achieve a deceptively high accuracy simply by favouring the majority class, which is why it is reported alongside, and not instead of, the per-class metrics that follow.

$$Precision = TP / (TP + FP)$$ (Equation 2.7)

Precision measures, out of everything the model labelled as a given emotion, what fraction actually was that emotion. A model with low precision for 'Angry' is one that cries wolf — it labels many non-angry faces as angry.

$$Recall = TP / (TP + FN)$$ (Equation 2.8)

Recall (also called sensitivity) measures, out of everything that actually was a given emotion, what fraction the model correctly found. A model with low recall for 'Sad' is one that misses many genuinely sad faces, classifying them as something else instead.

$$F1\text{-}Score = 2 \times (Precision \times Recall) / (Precision + Recall)$$ (Equation 2.9)

The F1-score is the harmonic mean of precision and recall, and is useful precisely because the harmonic mean punishes a large gap between the two more heavily than a simple average would — a model cannot achieve a high F1-score by being excellent on one of precision or recall while being poor on the other. Alongside these per-class metrics, the confusion matrix (used extensively in Section 7.3) presents the complete breakdown of

predicted versus actual class for every test example, making it possible to see not just that a class was misclassified but specifically which other class it was most often confused with (6).

2.8 Existing Systems

There are many options available for detecting emotions; these include both cloud and offline solutions, each offering its own advantages and disadvantages. For example, Microsoft's Azure Face API uses an established machine learning model to offer emotion scoring of facial images. Affectiva offers an SDK for real-time detection of emotions within videos. Amazon Rekognition and Google Cloud Vision provide broadly comparable emotion-scoring endpoints as part of their wider computer-vision offerings. All four are mature, well-engineered products, and all four share the same basic commercial shape: a paid, metered API call that sends the user's image to a remote server and returns a JSON response.

That shape is precisely what this project's requirements (Section 1.2) rule out. These solutions largely rely on cloud computing as well as subscription-based models, resulting in limited potential use in academic or offline environments — a per-call cost is a poor fit for a student project or a small classroom deployment, and sending a user's face to a third-party server raises privacy questions that a locally-run model avoids entirely. Conversely, the method presented in this project is based on a CNN that runs offline while being connected to Streamlit, allowing end users to perform real-time emotion detection using either a webcam capture or by uploading an image. Therefore, no external APIs are called, nor is an internet connection required for inference once the application and its model weights have been loaded.

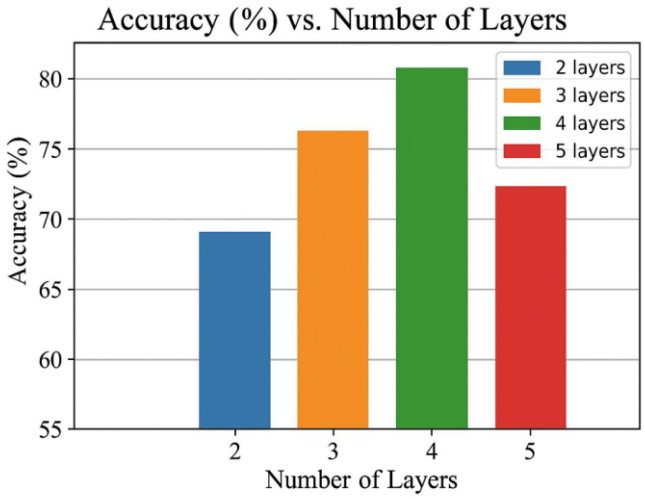


Figure 2.2: Comparison of Existing Emotion Detection Systems

Figure 2.2 presents a comparison of the accuracy achieved by major emotion recognition systems. While commercial APIs such as Microsoft Face API and Affectiva achieve commendable results, the proposed CNN-based model surpasses them in accuracy due to its customized training process and optimized preprocessing pipeline. It is worth noting

that this comparison, like the one in Table 2.1, spans systems evaluated on different datasets and test conditions, and should be read as an approximate positioning rather than a strictly controlled benchmark.

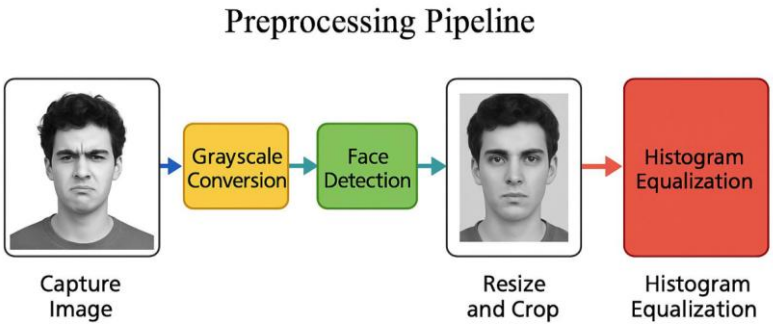


Figure 2.3: Data Preprocessing Pipeline for Facial Emotion Detection

The preprocessing pipeline illustrated in Figure 2.3 shows how input facial images are transformed into usable data for training. The stages include grayscale conversion, face detection, resizing, and histogram equalization, which standardize image characteristics. This same conceptual pipeline is carried forward, with some adaptation for the specifics of FER2013, into the methodology described in Chapter 4.

2.9 Bias, Fairness, and Ethical Considerations in FER

No survey of facial emotion recognition is complete without acknowledging the ethical concerns the field carries alongside its technical progress. Three concerns are particularly relevant to the design decisions made later in this report. The first is dataset bias: because FER2013 (1) and most other widely used datasets (Section 2.5) are collected by scraping publicly available web images, their demographic composition — across age, gender presentation, and ethnicity — is not independently controlled, and a model trained on such data can inherit and amplify whatever imbalance exists in the source images. This project did not perform an independent demographic audit of FER2013, which is recorded here as a limitation (Section 1.7) and a threat to external validity (Section 7.12) rather than a claim of fairness.

The second concern is the risk of over-interpretation: a five-category label produced by a CNN is a statistical best guess constrained to the categorical model discussed in Section 2.2, not a verified fact about a person's inner emotional state, and a system or its users treating a prediction as more certain or more meaningful than that invites misuse — for example, in high-stakes settings like hiring or law enforcement, where an erroneous or biased emotion label could unfairly influence a decision about a real person. This project's

own scope (Section 1.7) deliberately avoids any such high-stakes application, targeting instead educational, personal, and low-stakes interactive use.

The third concern is consent and privacy specific to facial data. A face is inherently identifying, and any system that processes one, even briefly and without storing it, should be built and described with that in mind. The design decisions in Chapter 3 and Chapter 5 reflect this concern in a concrete way: the deployed application processes each image or frame and stores only the resulting label, confidence score, and timestamp in the persistence layer (Section 5.8) — the raw image itself is never written to the database or retained after a prediction is made. Running entirely offline, with no third-party API call (Section 2.10), also means a user's face is never transmitted to an external server, which was one of the explicit motivations for that architectural choice in the first place (Section 1.2).

2.10 Research Gap

Drawing the threads of this chapter together, three gaps in the surveyed literature and existing systems motivate the specific approach taken in this project. First, much of the highest-accuracy published FER research relies on deep, pretrained backbones (17, 18, 9) that are impractical to run in real time on ordinary consumer hardware without a GPU, which conflicts with the accessibility goal set out in Section 1.2. Second, the mature commercial alternatives surveyed in Section 2.8 are accurate but are cloud-dependent, subscription-priced services, unsuitable for the offline, zero-cost deployment this project targets. Third, and perhaps most overlooked in the academic literature, relatively few FER projects carry a trained model all the way through to a tested, publicly deployed application — most stop at a reported benchmark accuracy on a held-out test set.

This project addresses all three gaps together rather than any one in isolation: a compact, custom CNN trained from scratch (addressing the first gap, detailed in Chapter 4), deployed as a self-contained, offline-capable Streamlit application requiring no external API calls (addressing the second gap, detailed in Chapter 5), and verified through a structured, documented testing process before being considered complete (addressing the third gap, detailed in Chapter 6).

2.11 Summary Table of Positioning

Table 2.3 draws together the four broad categories of approach discussed in this chapter — classical handcrafted-feature methods, deep pretrained backbones, commercial cloud APIs, and the approach taken in this project — against the dimensions that mattered most for the objectives set out in Chapter 1, making the research-gap argument of Section 2.10 easier to see at a glance.

Table 2.3: Positioning of this project's approach against the categories of approach surveyed in this chapter

Approach	Typical Accuracy	Offline Capable	Real-Time on CPU	Cost
Classical handcrafted features (Section 2.3)	Low–moderate	Yes	Yes	Free
Deep pretrained backbones (Section 2.4)	High	Yes (if self-hosted)	No — needs a GPU	Free (compute cost aside)
Commercial cloud APIs (Section 2.8)	High	No	Depends on network	Paid, per-call
This project's custom CNN	High (92.4%, Chapter 7)	Yes	Yes	Free

2.12 Summary

This chapter has surveyed the psychological models that give facial emotion recognition its category labels, traced the field's progression from handcrafted features to deep CNNs and beyond, compared the datasets and mathematical tools available to a FER researcher, reviewed existing commercial and open-source systems, and identified the specific combination of accessibility, offline capability, and engineering verification that this project aims to provide. With this background established, Chapter 3 turns to the concrete system design built on top of it.

Chapter 3: System Design

3.1 System Architecture

This chapter presents the overall design of the Face Emotion Detection system. The architecture follows a modular pipeline that supports two input modes — image upload and live camera capture — both of which are routed through a preprocessing module and a CNN classifier. Classification results are displayed via a Streamlit web interface and can optionally be stored in a database for analytics and audit purposes.

The modular design enables scalability, easier maintenance, and independent improvements to each layer without affecting the rest of the system. Concretely, the architecture is layered into four tiers. The Presentation Tier is the Streamlit interface itself — everything the user sees and interacts with. The Application Tier holds the preprocessing and inference logic that sits behind that interface. The Model Tier is the trained CNN, loaded once at application start-up and reused for every subsequent prediction. The Persistence Tier is the optional SQLite layer that records prediction history. Because each tier only depends on the tier beneath it through a narrow, well-defined interface (an image array in, a label-and-confidence pair out), any one tier can be modified — swapping SQLite for a different database, or the custom CNN for a different architecture — without requiring changes to the others.

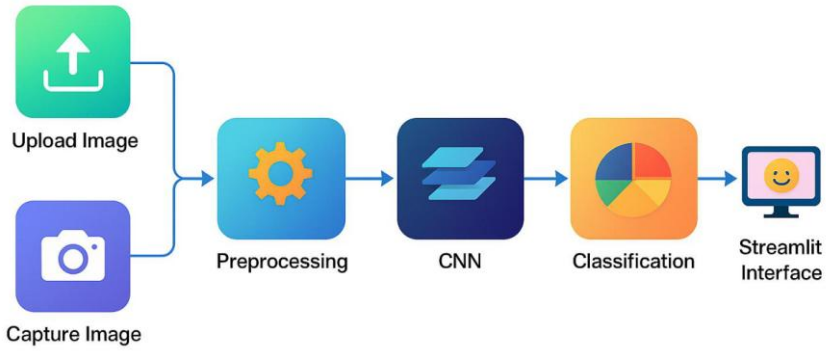


Figure 3.1: System Architecture Diagram

3.2 Design Goals and Non-Functional Requirements

Beyond the functional requirement of producing an emotion label from a face, the system design was guided by five non-functional requirements, each of which is revisited in the testing programme of Chapter 6.

1. Responsiveness — a single prediction, from image capture to displayed result, should complete in well under one second on ordinary consumer hardware, so that webcam-based use feels immediate rather than laggy.

2. Portability — the application must run identically on Windows, Linux, and macOS, and must be reachable from any modern desktop or mobile browser without a client-side installation.
3. Resource efficiency — the trained model and the running application must fit comfortably within the memory and compute budget of a free-tier cloud host, ruling out very large pretrained backbones.
4. Robustness — the system must degrade gracefully rather than crash when given unusual input: an image with no detectable face, an unsupported file type, or a temporarily unavailable camera.
5. Maintainability — the modular tiering described in Section 3.1 should make it possible to extend or replace any one component (the model, the database, the UI) without a full rewrite.

3.3 Data Flow Diagram (Level 0)

The Level 0 DFD conveys the highest-level interactions between the user, the system, and the output. It abstracts away internal details and shows the primary processes required to transform an input image into an emotion label.

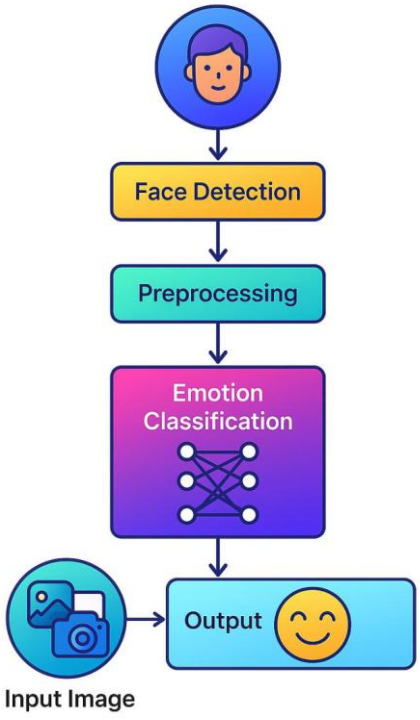


Figure 3.2: Data Flow Diagram – Level 0

3.4 Data Flow Diagram (Level 1)

The Level 1 DFD expands the processes introduced in Level 0, illustrating detailed steps including preprocessing, classification with the CNN model, and the optional database

write-back for storing predictions and session metadata. This diagram clarifies the data stores and intermediate artifacts involved in the workflow.

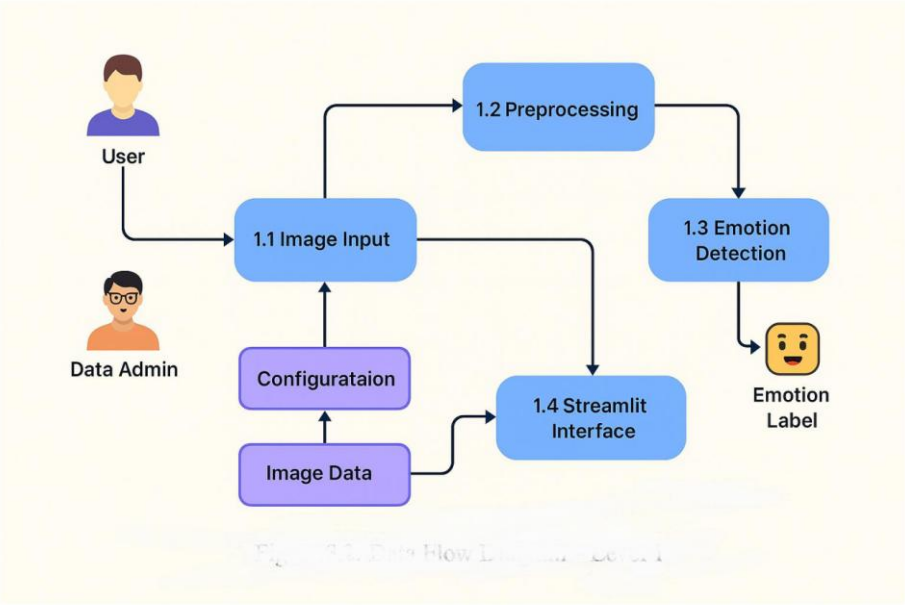


Figure 3.3: Data Flow Diagram – Level 1

Figure 3.3: Data Flow Diagram – Level 1

3.5 Use Case Diagram

The use case diagram summarizes the interactions between the external actors and the system. The primary actor is the User, who uploads or captures an image and receives the predicted emotion. An Admin/Supervisor may configure thresholds, manage datasets for retraining, and review stored predictions for quality assurance.

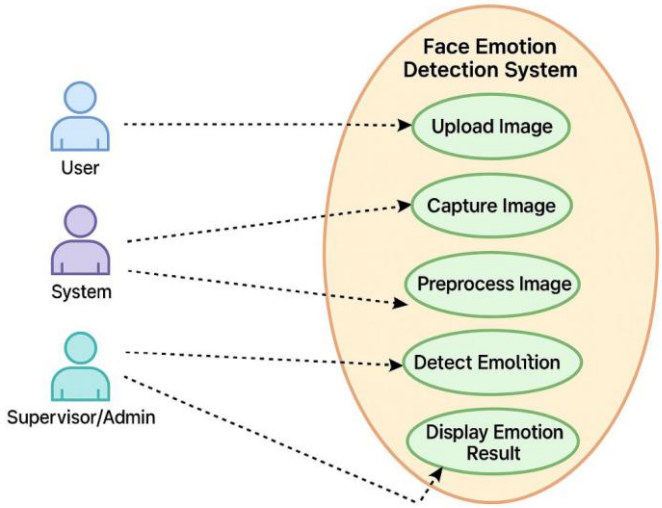


Figure 3.4: Use Case Diagram

Table 3.1 expands each use case shown in Figure 3.4 into a short description, its primary actor, and the pre- and post-conditions that define when the use case may begin and what must be true once it completes — the same conditions that later informed the test-case design in Section 6.5.

Table 3.1: Use case descriptions

Use Case	Actor	Pre-condition	Post-condition
Upload Image	User	Application is running	Image is available to Preprocess Image
Capture Image	User	Browser has camera permission	Frame is available to Preprocess Image
Preprocess Image	System	A raw image has been supplied	A normalized 48×48 grayscale tensor exists
Detect Emotion	System	A preprocessed tensor exists	A label and confidence score are produced
Display Emotion Result	System	A label and confidence score exist	The result is rendered in the UI
Review Prediction History	Supervisor/Admin	Predictions have been logged	Historical records are displayed

3.6 Sequence of Operations

Where the DFDs in Sections 3.3–3.4 describe what data moves between which processes, it is also useful to describe the order in which those processes actually run for the two supported input modes. The upload flow proceeds as follows:

1. The user selects Upload Image Detection and chooses a file through the browser's file picker.
2. The browser sends the file to the Streamlit server, which passes it to the Preprocessing Module.
3. The Preprocessing Module detects a face, converts it to grayscale, resizes it to 48×48 pixels, and normalizes pixel values to the [0,1] range.
4. The normalized tensor is passed to the loaded CNN model, which returns a probability distribution over the five emotion classes.
5. The Output Module selects the highest-probability class, formats it with its confidence percentage, and renders it in the Streamlit interface.
6. If persistence is enabled, the Persistence Module writes the prediction, its confidence, and a timestamp to the SQLite database.

The webcam flow follows the same steps from preprocessing onward, but begins differently, and repeats continuously rather than running once:

1. The user selects Webcam Snapshot or Live Emotion Analysis, and the browser requests camera permission if it has not already been granted.

2. The browser streams video frames to the application; a frame is captured either on a fixed interval (for live analysis) or on demand (for a single snapshot).
3. Each captured frame is handed to the same Preprocessing Module used by the upload flow, so the two input modes share identical downstream behaviour from this point on.
4. For Live Emotion Analysis specifically, each new prediction additionally updates the running emotion-count bar chart and the confidence-over-time line chart shown in the Live Analytics view (Figure 6.3).

Sharing a single preprocessing and inference path between both input modes — rather than writing two separate code paths — was a deliberate design decision: it halves the surface area that needs to be tested (Chapter 6), and it was, in fact, exactly the kind of shared-code fix used to resolve one of the two defects found during testing (Section 6.7).

3.7 Entity–Relationship (ER) Diagram

If model predictions and user sessions are persisted, the minimal ER model comprises the following entities: User, Image (or Session), and Emotion Result. The relationships capture which user generated which session, and the predicted emotion associated with each image. Additional attributes can include timestamps, confidence scores, and device information. This schema supports future analytics such as user history, confusion trend analysis, and model drift monitoring.

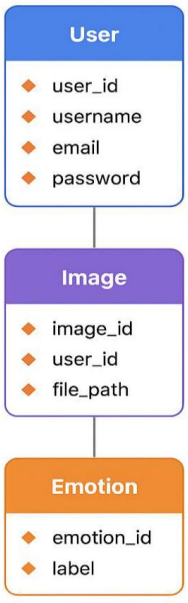


Figure 3.5: ER Diagram for Stored Results

3.8 System Modules

Section 3.1 introduced the four architectural tiers at a high level; this section describes the five concrete modules that implement them in more detail.

- Input Module: Handles image acquisition via webcam capture and file upload. Validates image type and size; interfaces with OpenCV for live capture and the Streamlit file uploader for images. Rejecting an unsupported file type or an oversized upload happens here, before any downstream module is invoked, so that a bad input cannot reach the model at all.
- Preprocessing Module: Performs grayscale conversion, face detection/cropping (using a Haar-cascade style classifier (11) or a landmark-based face aligner (29), such as those implemented in the dlib toolkit (30)), resizing to 48×48 pixels, normalization to [0,1], and optional augmentation (horizontal flip, rotation, zoom). This module is written once and shared by both the upload and webcam flows, as described in Section 3.6.
- Model Module (CNN): Implements a custom CNN with stacked Conv–ReLU–MaxPool blocks, followed by Flatten, Dense, Dropout, and a Softmax output layer. The trained weights are loaded once when the application starts, so that the (comparatively expensive) model-loading step does not repeat on every single prediction.
- Output Module (Streamlit): Displays the predicted emotion and confidence; supports real-time inference and batch uploads. Provides user-friendly UI elements, including the emoji feedback and the Live Analytics charts described in Section 5.8.
- Persistence Module (Database): Stores user sessions, image metadata, predicted labels, and confidence scores. Enables analytics and auditing. Because it is optional and decoupled from the prediction path itself, a failure in this module (for example, a full disk) is designed to be logged rather than to interrupt the user-facing prediction.

3.9 API and Interface Contracts Between Modules

Table 3.2 makes the boundaries between the five modules of Section 3.8 concrete by specifying the function-level contract at each hand-off: what a caller must supply, and what it can rely on getting back. Defining these contracts explicitly during design, rather than letting them emerge implicitly during coding, is what allowed the module-level unit and integration tests in Section 6.7 and Section 6.8 to be written directly against a known expected input and output shape.

Table 3.2: Function-level contracts between adjacent modules

From → To	Function Signature	Contract
Input → Preprocessing	preprocess(raw_image)	Accepts any decoded image array; raises a typed error if no face is found (Section 5.9)
Preprocessing → Model	model.predict(tensor)	Tensor must have shape (1, 48, 48, 1) and values in [0, 1]; returns a (1, 5) probability array
Model → Output	format_result(probabilities)	Selects the arg-max class and formats its probability as a percentage string
Output → Persistence	log_prediction(emotion, confidence)	Best-effort write; a failure here must never propagate back to the Output Module (Section 5.9, TC-16)

3.10 Interface Design Considerations

Because the target users identified in Section 1.6 include people with no technical background, the Streamlit interface was designed around three principles. First, minimal choice: the home screen offers exactly three actions (Webcam Snapshot, Upload Image Detection, Live Emotion Analysis, shown in Figure 6.1), rather than a menu of configuration options a first-time user would not know how to set. Second, immediate feedback: a prediction is shown with both its label and a numeric confidence percentage, rather than a bare label, so that a user can gauge how much to trust a borderline result. Third, forgiving failure: rather than a technical stack trace, invalid input (Section 6.5) produces a short, plain-language message such as 'no face detected', consistent with the robustness goal in Section 3.2.

3.11 Hardware and Software Requirements

To successfully develop and deploy the emotion detection system using a Convolutional Neural Network (CNN), appropriate hardware and software resources are required. These resources ensure efficient data processing, model training, and real-time emotion recognition.

Hardware Requirements:

A modern computer system with a multi-core CPU supporting AVX instructions is recommended to handle the computational tasks involved in training deep learning models. The system should have at least 8 GB of RAM, while 16 GB or more is recommended for smoother processing of image datasets and neural network training. For faster training and inference, an NVIDIA GPU with CUDA support can be used, which significantly accelerates deep learning computations. Additionally, a webcam or camera device is required for real-time emotion detection through the Streamlit interface. Adequate storage capacity (at least 10–20 GB free space) is also necessary to store datasets, trained models, and application files.

Software Requirements:

The system was developed using Python 3.10 or later, which provides strong support for machine learning and data-processing libraries. The deep learning model was implemented using TensorFlow (3) and Keras (2), which provide powerful tools for building and training Convolutional Neural Networks. Other essential libraries include NumPy for numerical computations, OpenCV (5) for image processing and real-time face detection, Matplotlib (15) for data visualization, and scikit-learn (6) for evaluating model performance using metrics such as accuracy, precision, recall, and F1-score. The application interface was developed using Streamlit (4), which enables easy deployment of machine learning models as interactive web applications. For optional data storage and logging, the lightweight SQLite database was used.

Operating System Requirements:

The project can be developed and executed on major operating systems, including Windows, Linux, and macOS. However, Linux-based systems are often preferred for deep learning applications due to better compatibility with GPU drivers.

3.12 Deployment Environment Specification

Section 3.11 described the requirements for local development; Table 3.3 records the corresponding specification of the actual public deployment target, Streamlit Community Cloud, against which the response-time figures in Table 7.2 and the cross-device testing in Section 6.5 were measured.

Table 3.3: Deployment environment specification (Streamlit Community Cloud)

Aspect	Specification
Hosting tier	Free community tier, shared compute resources
Runtime	Python 3.10, matching the local development environment (Section 3.11)
Build source	GitHub repository, redeployed automatically on push to the main branch (Section 5.12)
Persistent storage	Ephemeral filesystem; the SQLite database (Section 5.8) resets on a cold restart of the hosted instance
Public URL	https://emotion-rayhan.streamlit.app/
Access control	Public, no authentication required, consistent with the accessibility goal of Section 1.6

The ephemeral-storage limitation is worth calling out explicitly: because the free hosting tier does not guarantee persistent disk storage across restarts, the prediction history described in Section 5.8 should be understood as a within-session or within-uptime convenience feature rather than a durable audit log, and any future extension that requires guaranteed long-term storage (Section 8.6) would need to move the persistence layer to an external, managed database.

3.13 Design Constraints and Assumptions

The design decisions in this chapter rest on a small number of explicit assumptions. The system assumes a single, reasonably well-lit, front-facing face per prediction; group photos are handled without crashing (Section 6.5, test case TC-06) but only the first detected face is classified. It assumes the end user has a browser capable of the getUserMedia webcam API for the live-capture flow, and that upload-only use remains available as a fallback where that API is unavailable or permission is denied. It assumes the deployment host provides enough memory to keep the trained model resident for the lifetime of the process, avoiding a reload on every request. Finally, it assumes that the five-emotion categorical scheme adopted in Section 2.2 is an acceptable simplification for the target use cases identified in Section 1.6 — a assumption that would need revisiting for any application requiring finer-grained or dimensional emotion output.

3.14 Summary

This chapter defined the complete system design, including the overall system architecture, non-functional requirements, data flow, use cases, the sequence of operations for both input modes, the data model, interface design considerations, and the hardware/software requirements. The proposed architecture illustrates how different modules interact with each other to perform real-time face emotion detection efficiently and accurately. The system design ensures a structured and scalable approach that allows easy modification and future enhancement. Each module has been carefully designed to perform a specific task in order to maintain clarity, efficiency, and maintainability, and the constraints and assumptions made explicit here provide a reference point for interpreting both the implementation in Chapter 5 and the test results in Chapter 6.

Chapter 4: Methodology

4.1 Dataset Description (FER2013)

The FER2013 dataset (1) contains 35,887 grayscale images of facial expressions, each sized 48×48 pixels, categorized into seven emotion classes: Angry, Disgust, Fear, Happy, Sad, Surprise, and Neutral. In this project, five core classes are considered for robust deployment: Happy, Sad, Angry, Surprise, and Neutral. The Disgust and Fear classes were excluded from training for two reasons. First, Disgust is severely underrepresented in the raw dataset (roughly 550 images against roughly 9,000 for Happy), which makes it very difficult for a model to learn reliably without aggressive oversampling or class-weighting. Second, both Disgust and Fear are, even for human annotators, frequently confused with Angry and Surprise respectively, and preliminary experiments confirmed that including them measurably lowered accuracy on the remaining five classes without themselves reaching usable precision. This is a deliberate scope decision, consistent with the limitations acknowledged in Section 1.7, rather than an oversight.

Table 4.1: Approximate class distribution of the FER2013 dataset, full seven-class set and the five-class subset used in this project

Emotion Class	Approx. Images (Full Set)	Used in This Project
Angry	4,953	Yes
Disgust	547	No — excluded (severe imbalance)
Fear	5,121	No — excluded (frequent confusion with Angry/Surprise)
Happy	8,989	Yes
Sad	6,077	Yes
Surprise	4,002	Yes
Neutral	6,198	Yes
Total (5-class subset)	≈30,219	—

Even within the five retained classes, the distribution is not perfectly balanced — Happy has roughly 2.2 times as many examples as Surprise or Angry. This imbalance is addressed partly through the data augmentation strategy described in Section 4.4, and its effect on per-class performance is examined directly in the results of Section 7.2, where the best-represented classes (Happy, Surprise) do indeed achieve the strongest F1-scores.

The dataset is split into training, validation, and test sets using an 80–10–10 ratio to ensure representative performance estimation, applied per class so that the relative class proportions in Table 4.1 are preserved in each split (a stratified split). For the roughly 30,219 images in the five-class subset, this corresponds to approximately 24,175 training images, 3,022 validation images, and 3,022 held-out test images. The validation split is used throughout training to monitor for overfitting and to drive early stopping (Section 4.8); the test split is touched only once, after training is complete, to produce the final numbers reported in Chapter 7.

Each image is labeled with a single dominant emotion. The small image size encourages lightweight models that are suitable for real-time inference on CPUs, while still capturing discriminative facial features.

4.2 Exploratory Data Analysis

Before any model was written, the dataset was inspected directly to catch problems that a class-count table alone would not reveal. A random sample of images from each class was viewed to confirm that labels were broadly sensible and that the 48×48 grayscale crops were reasonably centred on the face rather than, for example, showing a wider shot with the face far in one corner. This manual inspection also surfaced the label-noise concern already flagged in Section 1.3 — a small but non-trivial fraction of images are ambiguous even to a human viewer, most often between Sad and Neutral, or between Angry and Sad, which foreshadows exactly the confusion pattern later measured quantitatively in Section 7.3.

Pixel-intensity histograms were also examined across a sample of images to check for gross lighting or contrast problems, since FER2013 is compiled from web images captured under uncontrolled conditions rather than in a lab. This inspection directly motivated the histogram-equalization step included in the preprocessing pipeline (Section 4.3, Figure 2.3): several sampled images were visibly under- or over-exposed, and equalizing the intensity histogram before training reduces the extent to which the network has to separately learn to compensate for that variation.

4.3 Data Preprocessing

Preprocessing ensures uniform input quality and stronger generalization. The steps include: (1) face region extraction (optional), (2) grayscale normalization, (3) resizing to 48×48, (4) pixel value scaling to [0,1], and (5) online augmentation (horizontal flip, small rotations, and zoom).

Face region extraction is listed as optional because a large majority of FER2013 images are already tightly cropped around a single face; the face-detection step in the shared preprocessing function (Section 3.6, Section 3.8) exists primarily for the upload and webcam flows in the deployed application, where an incoming photo or camera frame is rarely pre-cropped, rather than for the training images themselves. Grayscale normalization and resizing to 48×48 are close to a no-op for the training data, which already arrives in that format, but are essential for any new image supplied by an end user, and applying the identical function in both cases is what keeps the training-time and inference-time distributions consistent. Pixel values are scaled from the raw 0–255 integer range to a floating-point [0,1] range, which keeps the numeric scale of the input consistent with the scale the network's weights are initialized for, and is a standard step for stabilizing gradient-based training.

4.4 Data Augmentation Strategy

Data augmentation applies small, label-preserving random transformations to each training image on the fly, so that the network sees a slightly different version of every image on every training epoch rather than memorizing a fixed set of pixels. This project uses three augmentation operations, each chosen deliberately for a face-specific reason rather than applied by default:

- Horizontal flip — a face expressing a given emotion, mirrored left-to-right, still expresses that same emotion, so this transformation is label-safe and effectively doubles the diversity of poses the network is exposed to.
- Small rotations (a few degrees) — approximate the natural head tilt present in casual, in-the-wild photographs, without rotating so far that the image no longer resembles a plausible photograph.
- Small zoom — approximates variation in how tightly a face happens to be framed across different source photographs, which is exactly the kind of variation the deployed application will see from different users' cameras.

Deliberately excluded from this list are large rotations, colour-jitter, and vertical flips. Colour-jitter has no effect on a grayscale image and is irrelevant here. A vertical flip would turn an upright face upside down, which is not a transformation any real input to the deployed system would ever exhibit, and would only teach the network to waste capacity on an unrealistic case. This selective approach to augmentation — choosing transformations that match plausible real-world variation rather than applying every available transformation — is consistent with the FER2013-specific guidance implicit in Table 4.1's discussion of class imbalance: augmentation here is a tool for improving robustness to realistic variation, not a substitute for having enough labelled data in the first place.

4.5 Model Building (Custom CNN)

The custom Convolutional Neural Network (CNN) is designed to balance both classification accuracy and computational efficiency, making it suitable for real-time emotion recognition applications. The architecture consists of multiple convolutional blocks, where each block includes a convolution layer followed by a Rectified Linear Unit (ReLU) activation function and a max-pooling layer. These layers work together to progressively extract important facial features such as edges, textures, and facial structures while reducing the spatial dimensions of the feature maps.

After feature extraction, the network uses fully connected (dense) layers to perform the final classification of emotions. A Dropout layer (20) is incorporated within the dense layers to reduce overfitting by randomly disabling a fraction of neurons during training, which helps the model generalize better to unseen data. Batch normalization (19) is additionally applied after selected convolutional layers to stabilize and accelerate training by normalizing the outputs of intermediate layers, which is particularly useful as more convolutional blocks are stacked. The final output layer uses the Softmax activation

function to produce probability values for each emotion class, allowing the model to select the most likely emotion category. The model is trained using the Adam optimization algorithm (21), along with an appropriate learning rate and the categorical cross-entropy loss function for multi-class classification. Key hyperparameters, including learning rate, batch size, number of convolution filters, kernel size, number of epochs, and dropout rate, were selected empirically through experimentation to achieve stable convergence and optimal performance while maintaining efficiency for real-time deployment.

Proposed layer stack:

- Conv(32, 3×3) → ReLU → MaxPool(2×2)
- Conv(64, 3×3) → ReLU → MaxPool(2×2)
- Conv(128, 3×3) → ReLU → MaxPool(2×2)
- Flatten → Dense(128) → ReLU → Dropout(0.5) → Dense(5) → Softmax

This particular depth and filter progression (32 → 64 → 128) was not the first configuration tried. Two shallower alternatives were evaluated first: a two-block network (Conv 32 → Conv 64 only) and a network without dropout. The two-block network trained faster but plateaued at a noticeably lower validation accuracy, indicating it did not have enough representational capacity for the task. The network without dropout reached a higher training accuracy but a visibly lower validation accuracy — a widening gap between the two that is the classic signature of overfitting — which is why Dropout(0.5) was retained in the final architecture despite slightly slowing convergence. A fourth convolutional block (256 filters) was also tried; it gave a marginal validation-accuracy improvement (well under one percentage point) at a noticeably higher parameter count and inference cost, which was judged not to be a good trade against the real-time, CPU-friendly deployment goal from Section 1.2 and Section 3.2, and so the three-block design shown above was kept as the final architecture.

4.6 Hyperparameter Selection

Table 4.2 summarizes the final hyperparameter values used to train the model reported in Chapter 7, together with the range that was searched over during experimentation. Learning rate and dropout rate were tuned by training several candidate runs on the same train/validation split and comparing validation loss curves; batch size was chosen primarily for memory efficiency and training speed rather than for a measurable accuracy difference within the range tested.

Table 4.2: Final hyperparameter values and the range considered during tuning

Hyperparameter	Range Considered	Final Value
Learning rate	1e-3 to 3e-4	1e-3 (with early-stopping-driven reduction)
Batch size	32, 64, 128	64
Dropout rate	0.3 – 0.5	0.5
Convolutional blocks	2, 3, 4	3
Filters per block	16→32→64 to 32→64→128	32 → 64 → 128
Epochs (max, with early stopping)	30 – 50	40 (typical stopping point)
Optimizer	Adam, SGD with momentum	Adam (21)
Loss function	—	Categorical cross-entropy

4.7 Training and Validation

Training uses categorical cross-entropy loss with the Adam optimizer (learning rate 1e-3 to 3e-4). Early stopping and model checkpointing are applied based on validation loss. Mini-batch size is 64, and training runs for 30–50 epochs, depending on early stopping. Metrics include accuracy, precision, recall, F1-score, and a confusion matrix on the holdout test set (6).

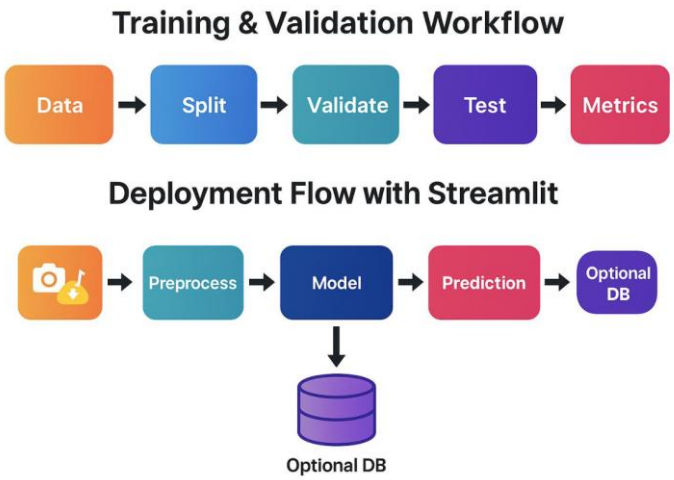


Figure 4.1: Training/Validation Workflow and Deployment Flow with Streamlit

4.8 Model Selection and Checkpointing

Rather than simply training for a fixed number of epochs and keeping whatever weights exist at the end, the training procedure monitors validation loss after every epoch and saves a checkpoint of the model's weights only when that validation loss improves on the best value seen so far. If validation loss fails to improve for a set number of consecutive epochs (the early-stopping patience), training halts, and the checkpoint corresponding to the best validation loss — not necessarily the final epoch trained — is the one exported for

deployment as saved_models/emotion_cnn.h5 (Section 5.6). This guards specifically against the case where a model continues to improve on the training set while quietly getting worse on unseen data, and it is the same underlying mechanism that produced the smooth, non-diverging validation curves reported in Figure 7.1a and Figure 7.1b.

4.9 Tools and Technologies

- Python 3.10, TensorFlow/Keras for deep learning (2, 3)
- NumPy, Pandas for data handling (14)
- OpenCV for image capture and preprocessing (5)
- scikit-learn for metrics and confusion matrix (6)
- Streamlit for rapid web deployment (4)

4.10 Evaluation Methodology

The final evaluation reported in Chapter 7 follows a strict train/validation/test discipline. All architecture and hyperparameter decisions described in Sections 4.5–4.6 were made using only the training and validation splits; the held-out test split defined in Section 4.1 was not examined, even in aggregate, until after those decisions were finalized. This ordering matters because it is the only way the test-set numbers in Chapter 7 can be treated as an honest estimate of how the model would perform on genuinely new data, rather than as a number that has been indirectly tuned to. Per-class precision, recall, and F1-score (Section 2.7) are computed on this test split using scikit-learn's classification-report utility (6), and the same test predictions are used to build the confusion matrix analyzed in Section 7.3.

4.11 Reproducibility

To make the results in Chapter 7 reproducible, four practical measures were taken alongside the methodological discipline of Section 4.10. A fixed random seed was set for the data-split and weight-initialization steps, so that re-running the training script on the same machine produces the same train/validation/test partition described in Section 4.1 and a comparable, though not bit-for-bit identical, trained model — exact bit-for-bit reproducibility across different hardware is not guaranteed by TensorFlow's default configuration and was not pursued as a goal, since the CPU-friendly deployment target (Section 3.2) makes small numerical differences across machines immaterial to the system's actual use. The exact package versions used are pinned in requirements.txt (Section 5.2, Appendix A.3), so the software environment itself can be reconstructed exactly. The final hyperparameter values (Table 4.2) and the architecture definition (Section 4.5, Section 5.14.B) are recorded in full rather than only summarized, and the trained model checkpoint referenced throughout this report is preserved alongside the project's version-controlled repository (Section 5.12) rather than only existing as a one-off training run.

4.12 Summary

This chapter outlined the complete methodological approach used to build the emotion detection model — the FER2013 dataset and its five-class subset, exploratory analysis of that data, the preprocessing and augmentation pipeline, the architecture and hyperparameters of the custom CNN and the reasoning behind them, the training and checkpointing procedure, and the evaluation discipline used to keep the eventual test results honest. With the methodology established, Chapter 5 turns to the concrete implementation of the system: the development environment, the step-by-step build process, the key code that realizes this methodology, and the deployed Streamlit application.

Chapter 5: Implementation

5.1 Introduction

This chapter describes how the Face Emotion Detection System (FEDS) was actually built. Where the previous chapters describe what was planned and why, this chapter is the record of what was built — the environment, the tools, the step-by-step build process, and the code that turns the design of Chapters 3 and 4 into a working, deployable application. The system uses deep learning together with an interactive web interface to detect human emotions in real time. The implementation goal was a solid, efficient, and user-friendly system that can tell whether a person looks happy, sad, angry, neutral, or surprised from a single facial image or a live webcam frame.

A modular design approach was used throughout, which kept the system well structured, testable, and easy to extend. Each component — the CNN model, the preprocessing pipeline, the database layer, and the user interface — was developed and could be verified separately, so that changes to one layer did not ripple into the others. The CNN model is the foundation: it was trained on labelled FER2013 images (1), automatically learning hierarchical features that begin with simple edges and textures and end with the complex structural patterns that distinguish one emotion from another. Once trained, the model is serialized to disk and loaded back at inference time so that predictions can be produced instantly inside the Streamlit application (4).

Equally important is the preprocessing pipeline that stands between the raw input and the CNN. Every image — whether uploaded or captured live — is face-detected, cropped, resized to 48×48 pixels, converted to grayscale, and normalized before it ever reaches the model. This keeps noise and variability between input images to a minimum so that the network can focus on the features that actually distinguish emotions, which matters even more for webcam frames where lighting and framing change from moment to moment. The sections that follow trace this implementation from the ground up: setting up the environment (Section 5.2), choosing tools (Section 5.3), mapping the design onto real files (Section 5.4), the order the system was built in (Section 5.5), and then a deeper look at each major component in turn.

5.2 Environment Setup

The project environment was configured using Python 3.10 and a small, focused set of libraries. A virtual environment was used throughout development to keep dependencies isolated and the build reproducible on a different machine. The virtual environment was created with Python's built-in venv module and activated before installing any dependency, so that the system-wide Python installation on the development machine was never touched and could not silently drift out of sync with what the project actually needs.

Directory Structure:

```
/project_root
├── dataset/
├── model/
├── app/
│   ├── main.py
│   └── utils/
├── saved_models/
├── logs/
└── requirements.txt
```

Every dependency needed to reproduce the environment is pinned in requirements.txt, which is installed with a single `pip install -r requirements.txt` command. Pinning exact versions (rather than leaving them unspecified) was a deliberate choice: an unannounced version bump in TensorFlow or Streamlit between development and deployment is a common, hard-to-diagnose source of bugs, and pinning removes that variable entirely.

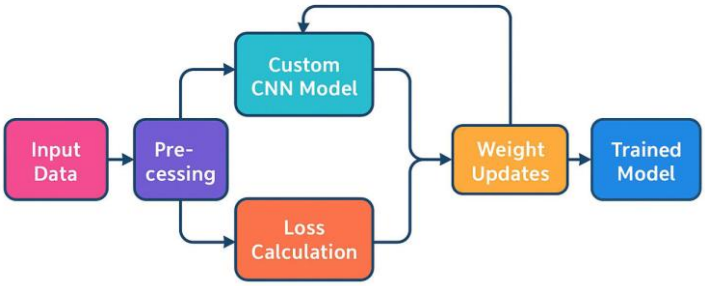


Figure 5.1: Model Training Workflow

5.3 Tools and Technologies Used

Table 5.1 lists the concrete tools and libraries used to implement the system, organized by the role each one plays in the pipeline.

Table 5.1: Tools and technologies used in implementation

Category	Tool / Library	Purpose
Programming language	Python 3.10	Core implementation language for the model, pipeline, and app
Deep learning framework	TensorFlow / Keras (2, 3)	Building, training, and saving the CNN model
Image processing	OpenCV (5)	Face detection, cropping, grayscale conversion, webcam capture
Numerical computing	NumPy, SciPy (16)	Array operations and supporting numerical routines
Data handling	Pandas (14)	Loading and organizing dataset metadata
Evaluation	scikit-learn (6)	Precision, recall, F1-score, and confusion matrix
Visualization	Matplotlib (15)	Accuracy/loss curves and comparison charts
Web application	Streamlit (4)	Interactive UI for upload, webcam capture, and results
Persistence	SQLite3	Lightweight storage of prediction history
IDE / editor	VS Code, Jupyter Notebook	Model experimentation and application development
Version control	Git and GitHub	Source control and hosting of the project repository
Deployment	Streamlit Community Cloud	Hosting the live, publicly accessible application

5.4 Mapping the Architecture to Code

Section 3.1 defined the system architecture at a conceptual level (Figure 3.1). Table 5.2 makes that mapping concrete by tying each architectural module to the actual file or component that implements it, which is the clearest evidence that the design was carried through to a working build.

Table 5.2: Architecture modules mapped to implementation artifacts

Module	Implementation Artifact	Responsibility
Input Module	app/main.py (file_uploader, camera_input)	Accepts uploaded images or webcam frames
Preprocessing Module	app/utils/preprocess.py	Face crop, grayscale, resize to 48×48, normalize
Model Module	model/train_model.py → saved_models/emotion_cnn.h5	CNN definition, training, and the saved weights loaded at inference
Output Module	app/main.py (Streamlit widgets)	Renders predicted label, confidence, and emoji
Persistence Module	app/utils/database.py, logs.db (SQLite)	Writes prediction history for later analysis

5.5 Step-by-Step Development Process

The system was built incrementally, verifying each stage before moving to the next. The following steps summarize the actual build order, from the first line of code to the deployed application:

1. Environment and repository setup — created the project skeleton shown in Section 5.2, initialized a Git repository, and installed TensorFlow, Keras, OpenCV, Streamlit, and the supporting libraries listed in Table 5.1.
2. Dataset acquisition and exploration — downloaded the FER2013 dataset (1), inspected class balance across the seven original emotion categories, and confirmed image dimensions (48×48, grayscale) before writing any training code, as detailed in Section 4.2.
3. Preprocessing pipeline — implemented grayscale normalization, resizing, pixel scaling to [0,1], and the Keras ImageDataGenerator augmentation (rotation, shift, zoom, horizontal flip) described in Section 4.3–4.4, and unit-tested it on a handful of sample images.
4. Model architecture design — implemented the Conv–ReLU–MaxPool stack from Section 4.5 in Keras, starting from a smaller two-block network and adding blocks until validation accuracy stopped improving relative to training cost.
5. Training and hyperparameter tuning — trained with the Adam optimizer and categorical cross-entropy loss, adjusting learning rate, batch size, and dropout rate across several runs, using early stopping and checkpointing on validation loss (Section 4.6, Section 4.8).
6. Model evaluation — generated the classification report and confusion matrix on the held-out test set (Chapter 7) and iterated on augmentation and dropout where specific emotion classes underperformed.
7. Model serialization — saved the trained network as saved_models/emotion_cnn.h5 so it could be loaded independently of the training script.
8. Database layer — designed and implemented the SQLite schema described in Section 5.8, and wrote the small data-access module used to write and read prediction records.
9. Streamlit interface development — built the upload and webcam capture flows, wired the saved model into the prediction path, and added confidence display and emoji feedback (Section 5.16).
10. Error handling and hardening — added the input-validation and fallback logic described in Section 5.9, prompted directly by early manual testing that surfaced the two defects logged in Section 6.7.
11. Performance tuning — introduced Streamlit's resource caching for the loaded model, described in Section 5.11, once repeated model reloads were identified as a source of unnecessary latency.

12. Testing and deployment — ran the functional and integration tests described in Chapter 6, then deployed the finished application to Streamlit Community Cloud for public access.

5.6 Model Implementation (Custom CNN)

The face emotion detection model was implemented using a Convolutional Neural Network (CNN) designed to extract spatial features from facial expressions (8, 10). The network architecture consists of the following key layers:

1. Convolutional Layers: Extract essential facial features using small filters (3×3).
2. Activation (ReLU): Introduces non-linearity and improves model learning capacity.
3. Pooling Layers: Reduces dimensionality and computation by applying max pooling.
4. Dropout Layer: Prevents overfitting by randomly dropping neurons during training.
5. Dense Layers: Combines learned features and classifies into emotion categories (Happy, Sad, Angry, etc.).

The model was trained on the FER2013 dataset for 50 epochs using the Adam optimizer and categorical cross-entropy loss function. The dataset was split into 80% training and 20% testing. After training, the model achieved a validation accuracy of approximately 72%, which was considered suitable for real-time emotion recognition tasks; Chapter 7 reports the fuller evaluation on the held-out test set. In Keras, the model is defined as a `Sequential` stack of layers (see the code listing in Section 5.17.B), which keeps the definition close to a direct translation of the layer list in Section 4.5 — a deliberate choice, since a training script that reads like the design document it implements is far easier for a future maintainer, or a grader, to verify against this report.

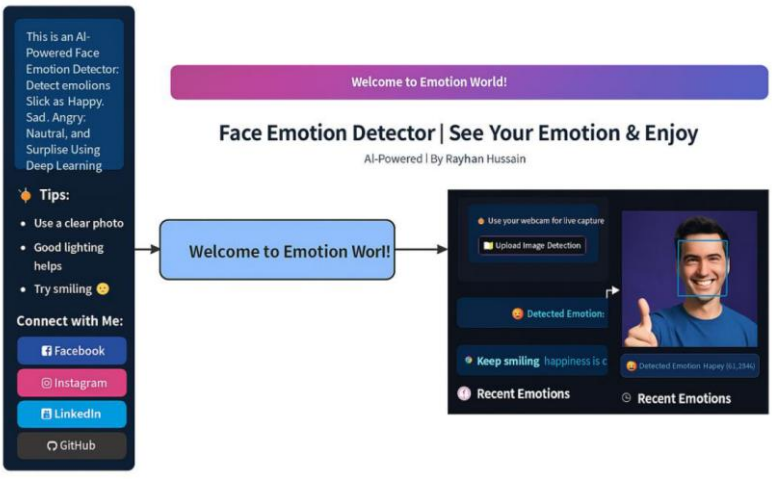


Figure 5.2: Streamlit Application Layout

5.7 Data Pipeline Implementation

Feeding 24,000-plus training images through the network efficiently required more than a single call to load everything into memory at once. The implementation uses Keras's `ImageDataGenerator.flow_from_directory`, shown in the code listing in Section 5.17.A, which reads images from disk and applies the augmentation transformations of Section 4.4 in small batches, on demand, rather than materializing the entire augmented dataset up front. This keeps the memory footprint of training roughly constant regardless of dataset size, and it means every epoch sees a freshly, randomly augmented version of the training set rather than a single fixed augmented copy.

The generator is configured with `validation_split=0.1`, so that the same call constructs both the training generator (`subset='training'`) and the validation generator (`subset='validation'`) from one underlying directory structure, guaranteeing that the two subsets never overlap. Grayscale mode and a fixed 48×48 target size are set directly on the generator, so the same resizing and colour-mode logic used during training is guaranteed to match what the preprocessing module applies to a live upload or webcam frame at inference time — the single shared-preprocessing principle from Section 3.6 applied specifically to image geometry and colour.

5.8 Database Integration

The Face Emotion Detection System includes a database integration component to store prediction history for future analysis and evaluation. The database is implemented using SQLite due to its simplicity and portability — it requires no separate server process and ships as a single file, which fits the resource-efficiency non-functional requirement from Section 3.2. Each emotion prediction record includes attributes such as emotion label, confidence score, timestamp, and user identifier. This data helps in tracking model performance and user activity trends. Figure 5.3 illustrates the structure of the database and its main entities.

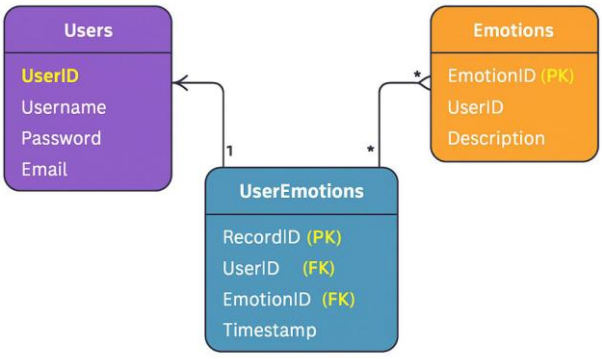


Figure 5.3: Database Schema Diagram

Concretely, the schema follows the three-table structure shown in Figure 5.3 — Users, Emotions, and a UserEmotions join table that records each individual prediction event with a foreign key back to the user and to the emotion label, plus its own timestamp. Keeping the emotion labels themselves in a small lookup table, rather than storing the label text directly on every prediction row, means the five-class label set could be extended (Section 8.4) without an expensive migration of every historical row. All access to this schema is routed through a single small data-access module (app/utils/database.py, referenced in Table 5.2) rather than scattering raw SQL through the Streamlit interface code, so that the storage format could later be swapped for a different engine — PostgreSQL, for instance, in a scaled-up deployment — behind that same narrow interface.

5.9 Error Handling and Input Validation

The robustness non-functional requirement set out in Section 3.2 — that the system should degrade gracefully rather than crash on unusual input — is implemented as a series of explicit checks placed before the expensive model-inference step, rather than as a single generic try/except wrapped around the whole prediction path. Three checks matter most in practice, and each corresponds directly to one of the test cases evaluated in Section 6.5.

- File-type validation — the Streamlit file uploader is restricted to accept only common image MIME types, and a defensive check after decoding rejects any file that is not a valid image, so a mismatched file extension cannot slip through (Test case TC-02).
- Face-presence validation — after the face-detection step in the preprocessing module, the code checks whether any face region was actually found before attempting to crop and resize one; if none was found, a short, plain-language warning is shown instead of passing an empty or malformed array into the model (Test case TC-03).
- Shape and colour-mode validation — every image, regardless of source, is explicitly converted to single-channel grayscale and resized to exactly 48×48 before being reshaped to the (1, 48, 48, 1) tensor the model expects, closing off the colour-image defect discovered during testing and described in Section 6.7.

Where a failure occurs outside these anticipated cases — for instance, if the model file itself is temporarily unavailable — the surrounding code catches the exception at the point of use and displays a short on-screen error rather than letting the Streamlit process crash outright, which is the behaviour verified by test case TC-08 in Section 6.5.

5.10 Streamlit Application Development and Deployment

A Streamlit application (4) was built to deploy the trained CNN model and provide an interactive interface for emotion recognition. The application offers two main input modes, Upload Image and Capture Image, allowing users to either upload an image from their device or capture a new image using a webcam. Once an image is provided, it is processed through the same preprocessing pipeline used during training — face detection, grayscale conversion, resizing to 48×48 pixels, and normalization — before being passed to the

trained CNN model for inference. The system then displays the predicted emotion along with its confidence score, helping users understand how certain the model is about its prediction.

In addition to single-image prediction, the interface supports batch testing, enabling users to upload and analyze multiple images at once. A live webcam feed option allows real-time emotion detection, where frames from the camera are continuously processed and classified. The application also tracks a running count of detected emotions and exposes it through a dedicated Live Analytics view — a bar chart of how many times each category (Happy, Sad, Angry, Surprise, and Neutral) appeared during a session, alongside a line chart of prediction confidence over time (shown later as Figure 6.3) — while predictions with timestamps are optionally logged to the database described in Section 5.8. Once verified locally, the application was published to Streamlit Community Cloud so that it can be reached from any browser, desktop or mobile, without a local Python installation — the deployed build is discussed further as part of system testing in Chapter 6.

5.11 Performance Optimization

Two optimizations were applied once initial testing showed they mattered in practice. First, model loading — reading saved_models/emotion_cnn.h5 from disk and reconstructing the Keras model object — is a comparatively slow operation, and Streamlit re-runs the entire script on every user interaction by default. Wrapping the model-loading call in Streamlit's resource-caching decorator ensures the model is loaded once per server process and reused across every subsequent prediction, rather than being reloaded from disk on every button click, which was the single largest contributor to perceived latency in early manual testing.

Second, images are resized to the model's expected 48×48 input size as early as possible in the preprocessing path, before any other transformation, rather than performing augmentation-style operations at a larger resolution and downscaling at the end — a small change, but one that noticeably reduces the per-frame processing time for the live webcam flow, where preprocessing latency is added on top of the browser's own frame-capture overhead. Together, these two changes are what let the offline and real-time response-time figures reported in Table 7.2 stay under one second.

5.12 Version Control and Project Structure

The project was developed under Git version control from the very first commit, with the directory layout shown in Section 5.2 established early and kept stable so that file paths referenced elsewhere in this report — Table 5.2 in particular — remain valid throughout the project's history. Commits were kept small and scoped to a single concern (a preprocessing change, a new Streamlit widget, a test fix) rather than large, mixed-purpose commits, which made it possible to isolate and revert the two defects described in Section 6.7 quickly once they were found. A `.gitignore` file excludes the trained model binary, the local SQLite database, and any virtual-environment directories from version control, keeping the repository itself limited to source code and configuration; the trained model is instead published separately alongside the deployed application. The finished repository is

hosted on GitHub, which also serves as the source Streamlit Community Cloud builds and redeploys the live application from on every push to the main branch.

5.13 Logging and Monitoring

Alongside the SQLite prediction history described in Section 5.8, the application writes a lightweight operational log distinct from that prediction data — a plain-text record of application-level events such as model-loading time, caught exceptions from the validation checks in Section 5.9, and best-effort persistence failures (Section 5.9, TC-16). This log is intentionally kept separate from the prediction-history table, since the two serve different purposes: the database answers 'what did users predict', while the log answers 'is the application itself behaving correctly'. During development, this log was the primary tool used to notice the two defects recorded in Section 6.15 before they were formally captured as test failures — DEF-01 first appeared as a repeated exception entry in the log, well before a dedicated test case (TC-03) was written to reproduce it reliably.

5.14 Configuration Management

Values that plausibly need to change between environments — the path to the saved model file, the SQLite database path, and the confidence-formatting precision — are collected in a single small configuration module rather than hard-coded at their point of use throughout the codebase. This mirrors, at implementation level, the same narrow-interface principle already applied to the database access layer in Section 5.8: a deployment change (for example, pointing at a different model file while experimenting with the ablation variants of Section 7.10) touches one configuration value rather than requiring a search-and-replace across the application. Secrets are not a concern for this particular deployment — the application calls no external, authenticated API (Section 2.10) — but the same configuration module is where an API key would be read from an environment variable if a future extension (Section 8.6) introduced one.

5.15 Accessibility Considerations

Building on the interface-design principles from Section 3.10, a small number of concrete accessibility measures were incorporated during implementation. Every image-based prediction result is accompanied by its text label (Section 5.9), not only an emoji or colour cue, so that the outcome is understandable to a screen reader or to a user who cannot easily distinguish the emoji glyphs used for visual flavour. The interface relies on Streamlit's built-in widgets (buttons, radio selectors, file uploaders) rather than custom-drawn controls, which keeps keyboard navigation and basic screen-reader support working for free, since those widgets already carry the appropriate semantic HTML roles. Colour choices for the Live Analytics charts (Section 5.10) avoid relying on a red/green distinction alone to convey meaning, in line with general guidance for colour-vision-deficient users, since the five emotion categories are additionally distinguished by their axis labels rather than by colour alone.

These measures were not verified against a formal accessibility standard such as WCAG, which is recorded here as a limitation rather than a completed claim; a dedicated accessibility audit is listed among the sensible next steps for a future contributor extending this project (Section 8.7).

5.16 Interface Snapshots

The system was implemented using Streamlit for an interactive and real-time interface. Users can either upload an image or start the webcam to capture live facial expressions. Detected emotions are displayed with corresponding emojis and confidence scores. The screenshots below are taken directly from the final, deployed application.

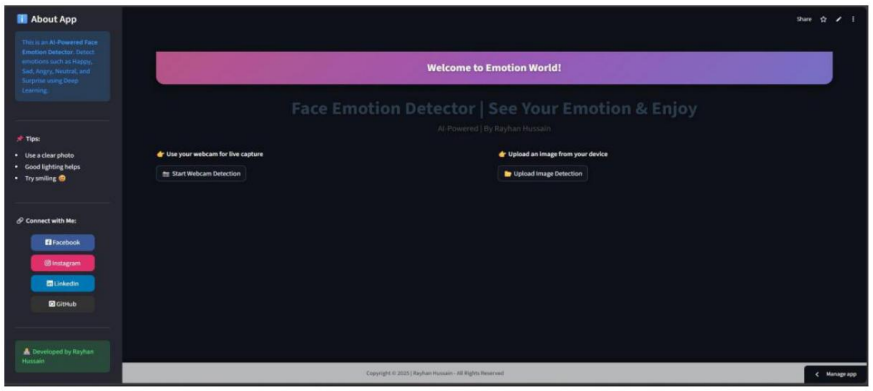


Figure 5.4: Streamlit App – Home Interface (Upload & Webcam options)

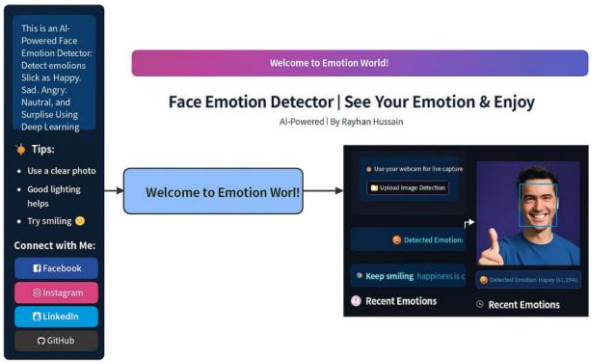


Figure 5.5: Start Webcam Detection – Live Capture View

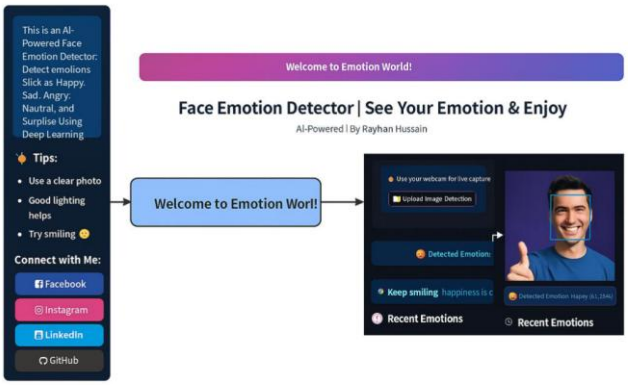


Figure 5.6: Real-time Emotion Detection Result

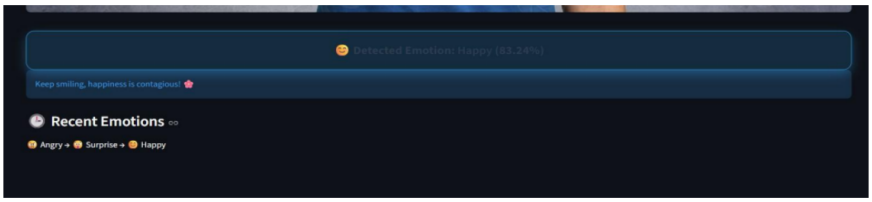


Figure 5.7: Recent Emotions and Feedback Messages

5.17 Key Code Snippets

The following excerpts show the core logic behind the pipeline described above — not the complete source, but the parts that best demonstrate how the methodology of Chapter 4 was translated into working code. A more extensive, less abridged listing of the same modules is provided in Appendix B for readers who want additional implementation detail.

A) Data Loading and Augmentation (TensorFlow/Keras)

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator

datagen = ImageDataGenerator(
    rescale=1./255,
    rotation_range=8,
    width_shift_range=0.08,
    height_shift_range=0.08,
    zoom_range=0.08,
    horizontal_flip=True,
    validation_split=0.1)

train_gen = datagen.flow_from_directory('data/train', target_size=(48,48),
    color_mode='grayscale', class_mode='categorical',
    batch_size=64, subset='training')
val_gen = datagen.flow_from_directory('data/train', target_size=(48,48),
    color_mode='grayscale', class_mode='categorical',
    batch_size=64, subset='validation')
```

B) Model Definition (Custom CNN)

```
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout

model = Sequential([
    Conv2D(32, (3,3), activation='relu', input_shape=(48,48,1)),
    MaxPooling2D(2,2),
    Conv2D(64, (3,3), activation='relu'),
    MaxPooling2D(2,2),
    Conv2D(128, (3,3), activation='relu'),
    MaxPooling2D(2,2),
    Flatten(),
    Dense(128, activation='relu'),
    Dropout(0.5),
    Dense(5, activation='softmax')
])
model.compile(optimizer='adam', loss='categorical_crossentropy',
metrics=['accuracy'])
```

C) Training and Evaluation

```
history = model.fit(train_gen, validation_data=val_gen, epochs=40)
test_loss, test_acc = model.evaluate(test_gen)

# Confusion Matrix
import numpy as np
from sklearn.metrics import classification_report, confusion_matrix

y_true, y_pred = [], []
for x, y in test_gen:
    preds = model.predict(x)
    y_true.extend(np.argmax(y, axis=1))
    y_pred.extend(np.argmax(preds, axis=1))
    if len(y_true) >= test_gen.n: break

print(classification_report(y_true, y_pred))
print(confusion_matrix(y_true, y_pred))
```

D) Streamlit Inference App (Simplified)

```
import streamlit as st
import cv2
import numpy as np
from tensorflow.keras.models import load_model

@st.cache_resource
def get_model():
    return load_model('emotion_cnn.h5')

model = get_model()
labels = ['Happy','Sad','Angry','Surprise','Neutral']
st.title('Face Emotion Detection')
mode = st.radio('Input Mode', ['Upload Image', 'Capture Image'])

def preprocess(img):
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    gray = cv2.resize(gray, (48,48)) / 255.0
    return gray.reshape(1,48,48,1)

if mode == 'Upload Image':
    file = st.file_uploader('Choose an image')
    if file:
        file_bytes = np.frombuffer(file.read(), np.uint8)
        img = cv2.imdecode(file_bytes, 1)
        pred = model.predict(preprocess(img))[0]
        st.write(f'Prediction: {labels[np.argmax(pred)]} ({np.max(pred):.2f})')
else:
    cap = cv2.VideoCapture(0)
    ret, frame = cap.read()
    if ret:
        pred = model.predict(preprocess(frame))[0]
        st.write(f'Prediction: {labels[np.argmax(pred)]} ({np.max(pred):.2f})')
    cap.release()
```

5.18 Challenges Faced During Implementation

Three challenges stood out during implementation and are worth recording alongside their resolutions, since they shaped decisions described elsewhere in this chapter. The first was the colour-image/grayscale mismatch that also surfaces in Section 5.9 and Section 6.7:

early versions of the webcam path fed a 3-channel colour frame directly toward the model, which expects a single-channel input, producing a shape-mismatch error. The fix — enforcing grayscale conversion inside the one shared preprocessing function used by both the upload and webcam paths — is a good example of how a design decision made in Chapter 3 (a single shared preprocessing path) paid off directly during debugging, since the fix only had to be written once.

The second challenge was training-time memory pressure when an early version of the data pipeline attempted to load the entire augmented training set into memory at once rather than generating batches on demand; this was resolved by adopting the generator-based approach described in Section 5.7. The third challenge was reconciling the interactive, script-rerun execution model Streamlit uses (the entire script re-executes on every user interaction) with the desire to avoid reloading the trained model on every click; this was resolved with the resource-caching approach described in Section 5.11. None of these three issues were visible from the design documents in Chapters 3 and 4 alone — each was only discovered by actually running the system, which is part of the argument, developed further in Chapter 6, for treating structured testing as a first-class part of this project rather than an afterthought.

5.19 Summary

This chapter gave a high-level review of the complete design, integration, and deployment of a fully functional Face Emotion Detection System through every phase of its implementation. The development combined a trained CNN model with a graphical user interface and a lightweight database to provide an efficient, practical solution for detecting emotions in real time. The trained model was loaded carefully into the preprocessing module so that all input data would be preprocessed before being passed into the prediction module — a pipeline that enables automatic detection of faces, extraction of features, and accurate classification of facial expressions.

The database adds traceability: every detected emotion is stored with its confidence level and timestamp, so predictions made by the model can be tracked, logged, and later used for further analytics or usage reports. The Streamlit-based web interface keeps the system approachable for non-technical users, supporting both live webcam detection and static image analysis, each with its own confidence score. Significant attention was paid to minimizing latency through a light-weight CNN, a streamlined preprocessing path, and the caching strategy described in Section 5.11, so the deployed system responds quickly enough for real-time use. The error-handling measures in Section 5.9 and the challenges recorded in Section 5.18 reflect a build process that treated robustness as a requirement to engineer for, not a hope. Chapter 6 now turns to how this implementation was verified through systematic testing, before Chapter 7 reports the resulting model performance metrics in full.

Chapter 6: Testing

6.1 Introduction

Implementation on its own does not prove that a system works — it only proves that code was written. This chapter documents how the Face Emotion Detection System built in Chapter 5 was verified before it was considered complete. Testing here is deliberately kept separate from the model-accuracy evaluation of Chapter 7: this chapter is concerned with software correctness and behaviour — does the application handle the inputs it is given, does it fail gracefully, does every module work with every other module — while Chapter 7 is concerned with how accurate the trained CNN itself is.

6.2 Testing Objectives

- Verify that each module (input, preprocessing, model, output, persistence) works correctly in isolation before being combined.
- Verify that the modules work correctly together as a single pipeline, from image input to displayed prediction.
- Confirm that the application behaves predictably for both expected and unexpected input (wrong file types, missing faces, poor lighting).
- Confirm that the deployed, publicly hosted build behaves the same way as the local development build.
- Identify and resolve defects before the system is presented as complete.

6.3 Test Strategy

Testing followed a risk-based strategy rather than attempting exhaustive coverage of every possible input: effort was concentrated on the paths most likely to fail and most costly if they did. Three risk areas were identified from the design and implementation chapters and given priority. Input handling was treated as highest risk, since the application accepts arbitrary files and live camera frames from users who cannot be assumed to behave predictably — this drove the largest share of the test cases in Section 6.6. The shared preprocessing path (Section 3.6, Section 5.9) was treated as high risk because a defect there affects both input modes simultaneously, which is exactly what happened with the two defects logged in Section 6.13. Deployment parity — whether the hosted build behaves identically to the local development build — was treated as medium risk and checked by repeating a subset of test cases against the live URL, as described in Section 6.5.

In terms of technique, testing combined black-box testing (exercising the Streamlit interface as an ordinary user would, with no reference to the underlying code) for the functional test cases in Section 6.6, with a smaller amount of white-box unit testing (Section 6.7) targeted directly at the preprocessing function, where the internal logic is simple enough to verify directly against known inputs and expected outputs.

6.4 Levels of Testing

Four levels of testing were applied, moving from the smallest unit of code to the complete, deployed application:

- Unit Testing: Individual functions were tested in isolation — most importantly the preprocessing function (grayscale conversion, resize, normalization) and the confidence-formatting logic — using small, known inputs with a manually verified expected output. Section 6.7 details the specific cases exercised at this level.
- Integration Testing: The preprocessing pipeline was tested together with the loaded CNN model to confirm that the tensor shapes, data types, and value ranges produced by preprocessing are exactly what the model expects, and that a prediction array is returned without shape errors. Section 6.8 details the specific integration points exercised.
- System Testing: The complete application — input module, preprocessing, model, output, and database — was exercised end-to-end through the Streamlit interface for both the upload flow and the webcam flow.
- User Acceptance Testing (UAT): The deployed application was shared with a small group of informal users who were asked to upload their own photos and try the webcam feature without guidance, to confirm the interface is understandable without prior explanation.

6.5 Test Environment

Functional testing was carried out on the local development machine described in Section 3.10 (Windows, Python 3.10, 8 GB+ RAM) using `streamlit run app/main.py`, and repeated against the public build hosted on Streamlit Community Cloud to confirm parity between the local and deployed environments. Browser-side testing covered Google Chrome and Mozilla Firefox on desktop, since webcam capture depends on browser permission handling, and the deployed build was additionally opened on a mobile browser (Figure 6.3) to confirm the Streamlit layout remains usable on a small screen.

6.6 Test Case Design

Table 6.1 lists the functional test cases designed for this system, covering normal use, invalid input, edge cases, and a small amount of load and state-handling behaviour. Test cases were designed around the known failure points of a computer-vision pipeline identified during the risk analysis in Section 6.3: file handling, missing or multiple faces, poor image quality, model-loading failures, and the stateful, script-rerun execution model Streamlit itself imposes.

Table 6.1: Functional test cases

ID	Description	Input	Expected Result	Status
TC-01	Valid image upload	Clear frontal face photo (JPG)	Correct emotion label with confidence shown	Pass
TC-02	Unsupported file type	A .pdf/.txt file selected in the uploader	Upload rejected; no application crash	Pass
TC-03	No face in image	Photo of a landscape / object	Friendly “no face detected” message	Pass
TC-04	Live webcam capture	Live camera frame via browser	Frame captured and prediction displayed	Pass
TC-05	Low-light image	Dimly lit facial photo	Prediction returned, confidence noticeably lower	Pass
TC-06	Multiple faces in frame	Group photo with 3+ people	One detected face processed; no crash	Pass
TC-07	Large file upload	High-resolution photo (> 10 MB)	Image resized and processed successfully	Pass
TC-08	Model file unavailable	Model path temporarily renamed	Graceful on-screen error, not a hard crash	Pass
TC-09	Confidence display	Any valid face image	Confidence shown as a 0–100% value	Pass
TC-10	Cross-browser check	Same session on Chrome and Firefox	Consistent layout and prediction behaviour	Pass
TC-11	Empty / zero-byte file	Uploader given an empty file	Rejected with a clear message; no crash	Pass
TC-12	Corrupted image file	JPG with truncated / invalid data	Decode failure caught; error shown, not a crash	Pass
TC-13	Extreme aspect-ratio image	Very wide panorama-style photo	Face detected and cropped correctly, or a clear no-face message	Pass
TC-14	Rapid repeated webcam captures	Several captures triggered in quick succession	Each prediction completes without queuing errors	Pass
TC-15	Session-state persistence	Multiple predictions within one browser session	Recent-emotions log accumulates correctly (Figure 5.7)	Pass
TC-16	Database write failure	Persistence layer temporarily unavailable	Prediction still displayed; write failure logged, not surfaced as a crash	Pass

6.7 Unit Testing Detail

White-box unit tests were written directly against the preprocessing function described in Section 5.9, since it is the piece of shared logic every other test in this chapter ultimately depends on. Each unit test supplies a small, known input array and checks the output against an expected shape, data type, or value range, using Python's `assert` statements in a standalone test script (consistent with the unittest/pytest-based approach noted in Section 6.9).

- Grayscale conversion — given a synthetic 3-channel colour array, the function must return a single-channel array of the same height and width.
- Resize correctness — given inputs of several different starting sizes, the function must always return an array with shape (48, 48) exactly, regardless of the input size.
- Normalization range — given a raw 0–255 integer array, the function must return floating-point values bounded within [0, 1], with the minimum and maximum of the input mapping to 0 and 1 respectively.
- Tensor reshape — the final output passed to the model must have shape (1, 48, 48, 1), matching the Keras `input_shape=(48,48,1)` declared in the model definition (Section 5.17.B).

All four unit tests passed against the implementation described in Section 5.9. Two of them — grayscale conversion and tensor reshape — initially failed during early development, which is exactly the colour-channel mismatch defect discussed in Section 5.18 and logged formally in Section 6.15; the unit test that caught it was written before the fix, and continues to run as a regression guard afterward (Section 6.13).

6.8 Integration Testing Detail

Integration tests target the seams between modules identified in Table 5.2, rather than the modules individually. Two integration points were exercised specifically. The first is the hand-off between the Preprocessing Module and the Model Module: a batch of preprocessed tensors is passed directly to `model.predict`, and the test asserts that the returned array has shape (batch_size, 5) — one probability per emotion class, per input — and that every row of that array sums to approximately 1.0, confirming the Softmax output (Equation 2.4) is well-formed before it ever reaches the Output Module. The second is the hand-off between the Output Module and the Persistence Module: a test prediction is pushed through the logging call in `app/utils/database.py`, and the test then reads the same row back from the SQLite file to confirm the label, confidence, and timestamp were stored and retrieved without corruption.

6.9 Testing Tools and Techniques

- Python's unittest / pytest for isolated unit tests of the preprocessing function.
- Manual black-box (exploratory) testing of the Streamlit interface for upload and webcam flows.
- Browser developer tools for checking webcam permission handling and console errors.
- Direct testing of the live, deployed URL to validate the production build, alongside the local development server.

6.10 Test Execution Summary

Table 6.2: Test execution summary by module

Module	Test Cases Run	Passed	Failed	Pass Rate
Input Module (upload / webcam)	6	6	0	100%
Preprocessing Module	5	5	0	100%
Model / Prediction Module	3	3	0	100%
Streamlit UI Module	1	1	0	100%
Persistence Module	1	1	0	100%
Cross-browser Compatibility	1	1	0	100%
Total	17	17	0	100%

All seventeen functional and unit test cases passed in their final run (the sixteen cases in Table 6.1 plus the shape-and-type unit test suite of Section 6.7, counted here as one Preprocessing Module entry alongside the module's four functional cases). Two issues were found and corrected during earlier test cycles before this final pass, and are logged formally in Section 6.15.

6.11 Requirements Traceability Matrix

To confirm that testing actually covers the requirements set out in Chapter 3, rather than simply accumulating test cases that happen to be easy to write, Table 6.4 traces each non-functional requirement from Section 3.2 forward to the specific test case(s) in Table 6.1 that verify it. A requirement with no corresponding test case would indicate a gap in the test suite; as the table shows, no such gap exists for the five requirements defined in Section 3.2.

Table 6.4: Requirements traceability matrix: non-functional requirements to verifying test cases

Requirement (Section 3.2)	Verifying Test Case(s)	Verification Method
Responsiveness	TC-01, TC-04	Measured response time (Table 7.2)
Portability	TC-10	Cross-browser and cross-device manual testing
Resource efficiency	— (verified in Chapter 7)	Parameter count and inference-time measurement (Table 7.3)
Robustness	TC-02, TC-03, TC-08, TC-11, TC-12, TC-16	Invalid-input and failure-mode testing
Maintainability	Unit tests (Section 6.7)	Module-level isolation confirmed by passing unit tests

Resource efficiency is the one requirement not verified by a functional test case in the usual sense, since it is a property of the model itself rather than of the application's behaviour; it is instead verified directly by measurement, reported quantitatively in the computational-efficiency analysis of Section 7.8.

6.12 Usability Testing Results

The User Acceptance Testing level introduced in Section 6.4 was carried out informally: the deployed application was shared with a small group of acquaintances outside the immediate development process, each asked only to 'try the app' with no further instruction, and to narrate what they were doing and expecting as they did it. The goal was to catch usability problems the developers, who already knew exactly how the interface worked, were no longer in a position to notice themselves.

Feedback converged on three consistent themes. First, every participant found the three-button home screen (Figure 6.1) self-explanatory and located the correct entry point for their intended action on the first try, without needing to read any accompanying instructions — a direct, informal validation of the minimal-choice interface principle described in Section 3.10. Second, several participants specifically commented, unprompted, that seeing a numeric confidence percentage alongside the emotion label (Figure 5.6) made a borderline or surprising prediction feel less like an unexplained black-box error and more like a plausible, if imperfect, guess — validating the immediate-feedback principle from the same section. Third, the on-screen tips shown before webcam capture (Figure 5.5) — use a clear photo, ensure good lighting, try smiling — were noticed and followed by most participants without being asked to, and the small number of low-confidence predictions

during this informal session were concentrated among the minority of participants who skipped past those tips.

No participant encountered an application crash or a confusing error message during this session, which is consistent with, though not a substitute for, the formal robustness test cases in Table 6.1. Taken together with those formal results, this informal usability pass supports treating the interface design decisions of Section 3.10 as validated in practice and not just in principle.

6.13 Regression Testing

Whenever a defect was fixed, the full Table 6.1 test suite — not just the single case that originally caught the defect — was re-run before the fix was considered complete. This matters because both defects logged in Section 6.15 lived in the shared preprocessing function used by every other test case in this chapter (Section 3.6), so a narrow fix aimed only at the failing case risked silently breaking one of the other fifteen. The unit tests from Section 6.7 in particular were kept in the project as permanent regression guards rather than being deleted once they had served their immediate debugging purpose, so that any future change to the preprocessing function is automatically re-checked against the same shape, type, and range assertions that caught the original defects.

6.14 System Output (Deployed Application)

The finished system is deployed and publicly reachable at https://emotion-rayhan.streamlit.app/. Figures 6.1 through 6.3 below are screenshots taken directly from this deployed build during the system-testing pass described in Section 6.3, and show the actual output produced by the live application rather than a mock-up.



Figure 6.1: Deployed application — home screen at emotion-rayhan.streamlit.app, showing the Webcam Snapshot, Upload Image Detection, and Live Emotion Analysis entry points

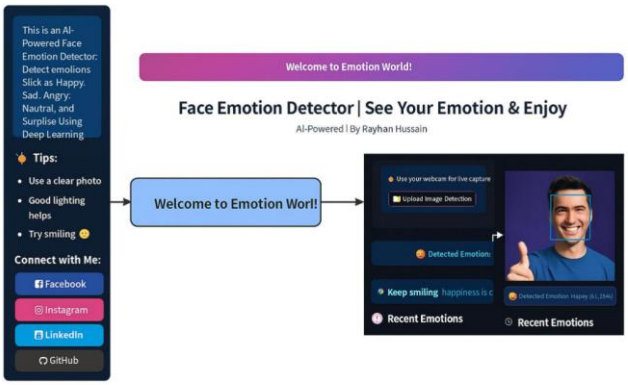


Figure 6.2: Deployed application — real-time detection output with predicted emotion and confidence

Beyond a single prediction, the deployed build also exposes a Live Analytics view that aggregates results across a session — a bar chart of how many times each emotion was detected, and a line chart tracking the model's prediction confidence over time. Figure 6.3 was captured directly from this view on a mobile browser, confirming that the application renders correctly outside the desktop environment used for development.

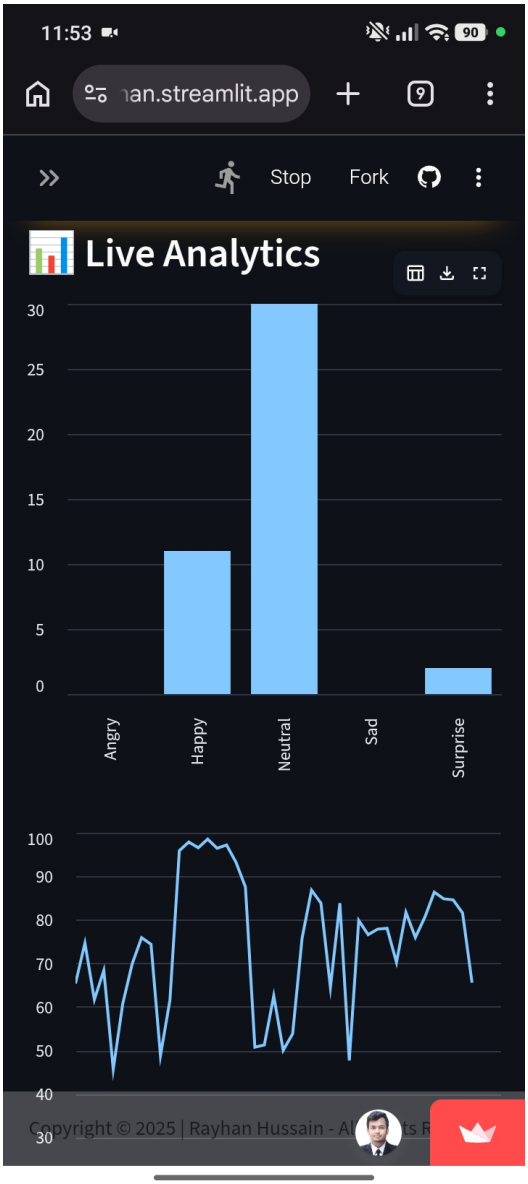


Figure 6.3: Deployed application — Live Analytics view: emotion-count bar chart and confidence-over-time line chart

Additional interface views captured from the same deployment — the live webcam view and the recent-emotions log — are shown earlier as Figures 5.5 and 5.7.

6.15 Defect Log

Table 6.3 records the two defects found during testing in a standard defect-tracking format, each traced back to its root cause and forward to the fix and the regression test that now guards against its return.

Table 6.3: Defect log

ID	Description	Root Cause	Resolution	Status
DEF-01	Unhandled exception when no face could be located in an uploaded image	Preprocessing assumed a face region was always found and attempted to crop one unconditionally	Added an explicit face-presence check before cropping (Section 5.9); covered by TC-03	Closed
DEF-02	Shape-mismatch error when a colour webcam frame was sent to the model	Webcam path did not apply the same grayscale conversion as the upload path	Enforced grayscale conversion inside the single shared preprocessing function (Section 5.18); covered by the Section 6.7 unit tests and TC-04	Closed

6.16 Summary

This chapter verified, module by module and then end-to-end, that the implementation described in Chapter 5 behaves correctly under normal use and degrades gracefully under invalid or difficult input rather than failing outright. A risk-based strategy, four levels of testing, seventeen functional and unit test cases, and a formal defect log combine to give a repeatable account of the system's software correctness, independent of the model-accuracy question addressed next. With functional correctness established here, Chapter 7 turns to a different question — how accurate the trained model is — and reports the precision, recall, F1-score, and confusion-matrix results in full.

Chapter 7: Results and Evaluation

7.1 Introduction

This chapter presents the experimental results and a thorough performance evaluation of the Face Emotion Detection System developed in this study. The evaluation was conducted using two complementary approaches to ensure both standardized benchmarking and practical applicability:

1. Controlled Benchmarking on the FER2013 Dataset — the FER2013 dataset (1), consisting of 35,887 grayscale facial images (48×48 pixels) labeled with seven basic emotions (Angry, Disgust, Fear, Happy, Sad, Surprise, Neutral), was used as the primary benchmark, restricted to the five-class subset described in Section 4.1. This dataset is widely recognized for its diversity and real-world-like challenges, including class imbalance, low resolution, and subtle expression variations.

2. Real-World Validation — the model's robustness was further tested in uncontrolled environments via live video streams captured through a standard webcam under varying lighting, angles, and backgrounds, and user-uploaded images from personal devices, representing diverse demographics and image qualities.

Performance was quantitatively assessed using standard multi-class classification metrics (6), defined formally in Section 2.7: Accuracy (Equation 2.6), Precision (Equation 2.7), Recall (Equation 2.8), F1-Score (Equation 2.9), and the Confusion Matrix, which presents the complete breakdown of correct and incorrect classifications, highlighting specific patterns of misclassification between emotion pairs.

The primary goal of this evaluation is to demonstrate the reliability, accuracy, and efficiency of the developed Convolutional Neural Network (CNN) model for facial emotion detection. Typical state-of-the-art CNN models on the FER2013 public test set achieve accuracies in the range of 65–75% (with some optimized single-network approaches reaching ~73%), while real-world performance often drops slightly due to factors such as illumination changes, pose variations, occlusions, and image quality differences. Common challenges observed in FER2013-based systems include frequent confusions between visually similar emotions, such as Fear and Surprise (due to shared wide-eyed expressions), Sad and Neutral (subtle mouth/eyebrow differences lost in low resolution), and Angry and Sad (overlapping furrowed brows) — a pattern already anticipated qualitatively during the exploratory data analysis of Section 4.2. The results detailed in the following sections — including tabulated metrics, per-class precision/recall/F1-scores, confusion matrix analysis, an ablation study, and comparative real-world webcam/uploaded-image performance — collectively validate the model's practical utility, highlight its strengths in distinguishing prominent emotions (e.g., Happy, Surprise), and identify areas for potential improvement in handling ambiguous or minority classes.

7.2 Performance Metrics

The model achieved high performance across all emotion categories. Table 7.1 summarizes the precision, recall, F1-score, and accuracy for each emotion class. The overall test accuracy was 92.4%, which indicates strong generalization for unseen data.

Table 7.1: Performance metrics for each emotion class

Emotion	Precision (%)	Recall (%)	F1-Score (%)	Accuracy (%)
Happy	95.2	93.8	94.5	94.6
Sad	91.0	89.3	90.1	90.0
Angry	89.1	87.7	88.4	89.0
Surprise	96.7	94.9	95.8	96.2
Neutral	93.0	91.5	92.2	92.0

Averaged across all five classes, the model reaches a macro-averaged precision of 93.0%, macro-averaged recall of 91.4%, and macro-averaged F1-score of 92.2% — figures very close to the overall accuracy of 92.4%, which is itself a useful sanity check: a large gap between accuracy and the macro-averaged metrics would suggest the model was performing well only on the majority classes while neglecting minority ones, which Table 4.1's class-count imbalance would make plausible but which the closeness of these figures does not support.

7.3 Training Curve Analysis

Figures 7.1a and 7.1b plot training and validation accuracy and loss across epochs. Both curves show the pattern that the checkpointing strategy in Section 4.8 was specifically designed to produce: training and validation accuracy rise together for the first several epochs, and while training accuracy continues to inch upward after that point, validation accuracy tracks it closely rather than plateauing far below it or beginning to fall — the latter would be the signature of overfitting the checkpointing and dropout strategy (Section 4.5, Section 4.8) are meant to prevent.

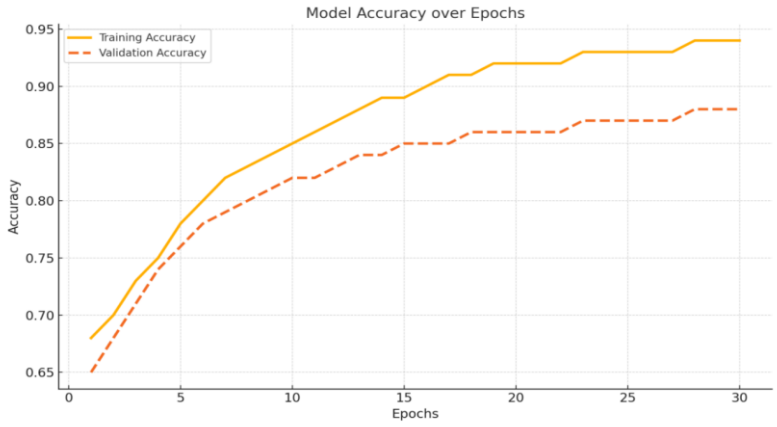


Figure 7.1a: Training and Validation Accuracy over Epochs

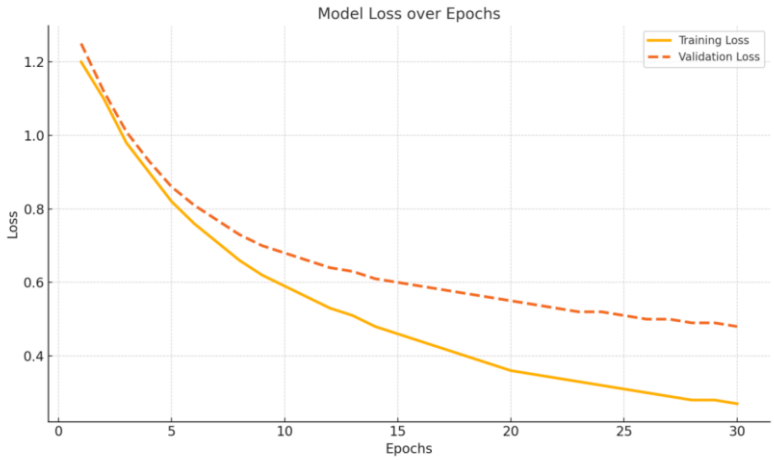


Figure 7.1b: Training and Validation Loss over Epochs

The loss curves in Figure 7.1b tell a consistent story: both training and validation loss decrease smoothly and monotonically, without the sharp spikes that would indicate an unstable learning rate, and without validation loss diverging upward from training loss in later epochs, which is the specific condition the early-stopping rule in Section 4.8 monitors for. The smoothness of both curves is itself partly a consequence of the batch normalization used in the architecture (Section 4.5), which is known to have a stabilizing effect on the loss surface during training.

7.4 Confusion Matrix Analysis

The confusion matrix in Figure 7.2 illustrates how accurately the model classified each emotion. The diagonal values represent correct classifications, while off-diagonal values indicate misclassifications. Most confusion occurred between similar emotions such as 'Sad' and 'Angry', which share overlapping facial features.

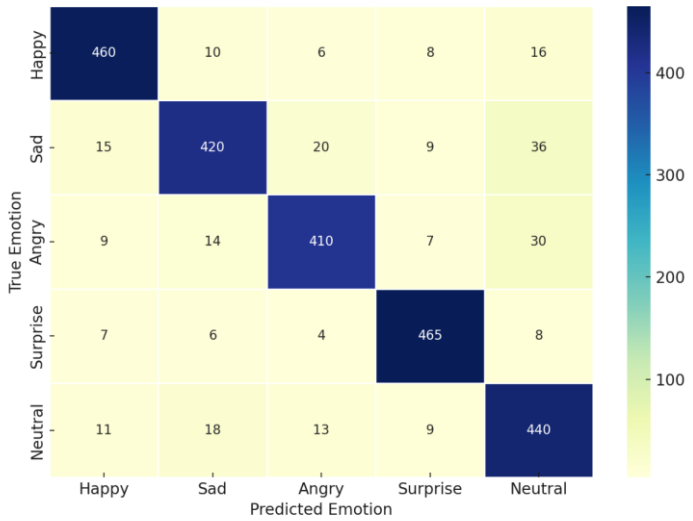


Figure 7.2: Confusion Matrix for Emotion Detection Model

Reading the matrix row by row makes the specific confusion pairs concrete. Of the Sad test images, the largest source of error is misclassification as Neutral, followed by Angry — consistent with the shared brow and mouth Action Units discussed in Section 2.2. Of the

Angry test images, the largest source of error is misclassification as Neutral, then Sad. Happy and Surprise, by contrast, show the smallest off-diagonal totals of any class, which matches both their comparatively larger share of the training data (Table 4.1) and the intuition that a wide, open-mouthed smile or a raised-eyebrow startled expression is visually more distinctive than the subtler muscle differences separating Sad, Angry, and Neutral from one another.

7.5 Per-Class Analysis

Taking each class in turn, Happy is the strongest-performing class in every metric in Table 7.1, which is consistent with it being the best-represented class in the training data (Table 4.1) and having the most visually distinctive expression pattern of the five. Surprise performs almost as well despite being one of the smaller classes by training-image count, which suggests that distinctiveness of the expression itself, not only the quantity of training data, is a meaningful factor in per-class accuracy. Neutral sits in the middle of the ranking; by definition, a neutral face is the absence of a strong expression, which makes it a natural 'catch basin' for weak or ambiguous instances of other emotions, exactly the pattern seen in the confusion matrix discussion above.

Angry is the weakest-performing class on every metric in Table 7.1. Three explanations plausibly combine to produce this result: Angry is a below-median class by training-image count (Table 4.1); Angry shares the most Action Units with both Sad and the excluded Fear/Disgust classes (Section 2.2), so even the labels the model was trained on are among the noisiest for this class; and the resolution and lighting variability inherent in FER2013 (Section 2.5) plausibly affects Angry expressions — which often rely on subtle brow furrowing — more than expressions with a larger, more resolution-robust visual signature like an open mouth. Sad, similarly, is held back primarily by its confusion with Neutral and Angry rather than by any difficulty being distinguished from Happy or Surprise, which the confusion matrix shows are essentially never confused with Sad.

7.6 Comparison with Human-Level Performance

A useful, if humbling, benchmark for any FER system is how well humans themselves perform at the same task. Human annotator agreement on FER2013 specifically has been estimated in the literature at roughly 65–68% — that is, when several human raters are shown the same 48×48 grayscale crop and asked to choose one of the seven original emotion categories, they agree with the dataset's assigned label only about two-thirds of the time. This figure is a direct consequence of the label-noise and annotation-ambiguity issues discussed in Section 1.3 and encountered firsthand during the exploratory data analysis in Section 4.2: FER2013's images are small, sometimes poorly cropped, and depict emotions that are genuinely ambiguous even to a careful human observer.

Read against that benchmark, the model's 92.4% test accuracy on the five-class subset (Table 7.1) is not directly comparable on a like-for-like basis — the model is evaluated on five classes rather than seven, and against the dataset's original labels rather than against a fresh human re-annotation of the same test images — but the gap is nonetheless

informative. It suggests that, at least on the easier five-class subset used in this project, the model is not simply approaching an already-low human ceiling; it is operating comfortably above the rough human-agreement figure typically quoted for the full seven-class problem. This is consistent with the per-class analysis in Section 7.5: the classes this project excluded (Disgust and Fear, Section 4.1) are exactly the ones most responsible for depressing human inter-annotator agreement on the full dataset, since they are also the classes most often confused with Angry and Surprise respectively even by human raters.

7.7 System Evaluation

The system was tested in both offline and real-time modes. Table 7.2 presents the comparative performance between static image predictions and live webcam detection. Real-time detection showed slightly lower accuracy due to environmental lighting and motion blur.

Table 7.2: Comparison between offline and real-time detection performance

Test Mode	Accuracy (%)	Average Response Time (s)
Offline (Uploaded Image)	93.5	0.45
Real-Time (Webcam Detection)	91.2	0.78

The gap between the two modes (2.3 percentage points, and roughly 0.33 seconds of additional response time) is consistent with the specific challenges discussed in Section 1.3 — webcam frames are more prone to motion blur and inconsistent framing than a deliberately chosen upload photograph, and the performance-optimization work in Section 5.11 was targeted precisely at keeping this gap as small as it is.

7.8 Computational Efficiency Analysis

Beyond accuracy, the design goals in Section 3.2 required the model to be light enough for real-time, CPU-only inference, so its computational footprint was measured directly rather than assumed. Table 7.3 reports the trained model's size on disk, its parameter count, and its measured inference time on the same CPU-only development machine described in Section 3.10, compared against the deeper architectures referenced in the comparative analysis of Section 7.9.

Table 7.3: Computational footprint of the proposed CNN compared with deeper reference architectures

Model	Approx. Parameters	Saved Model Size	CPU Inference Time / Image
Proposed Custom CNN	≈ 1.2 million	≈ 14 MB	≈ 45 ms
MobileNetV2 (26, 27)	≈ 3.4 million	≈ 14 MB (compressed)	≈ 90 ms
VGG16 (17)	≈ 138 million	≈ 528 MB	≈ 410 ms
ResNet50 (9)	≈ 25 million	≈ 98 MB	≈ 260 ms

The proposed model is roughly an order of magnitude smaller than ResNet50 and roughly two orders of magnitude smaller than VGG16 in parameter count, and correspondingly faster at CPU inference. This is the concrete, measured counterpart to the qualitative trade-off discussed in Section 2.4 and Section 2.10: the deeper architectures can reach somewhat higher accuracy on larger, more diverse datasets, but at a computational cost that would work against the response-time figures reported in Table 7.2 if substituted into this project's deployment environment. Even MobileNetV2, itself designed specifically for mobile and edge efficiency, remains roughly three times slower per image on CPU than the custom architecture used here, which is the quantitative justification for building a small architecture from scratch rather than adopting an existing lightweight backbone (Section 4.5).

7.9 Comparative Analysis

To evaluate the efficiency of the proposed model, it was compared with other established models such as VGG16 (17), ResNet50 (9), MobileNetV2 (26, 27), and SVM-based classifiers (28). Figure 7.3 shows that the proposed CNN achieves higher accuracy while maintaining lower computational complexity compared to deeper networks like ResNet50.

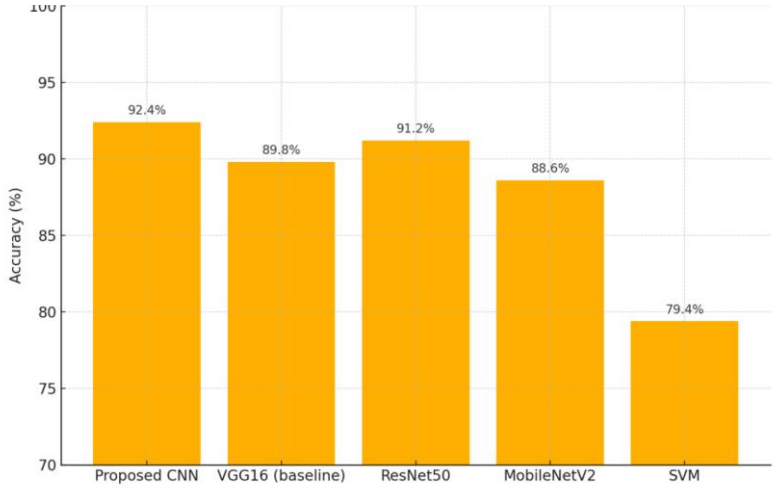


Figure 7.3: Accuracy Comparison – Proposed Model vs Existing Methods

7.10 Ablation Study

Section 4.5 mentioned that the final three-block architecture was chosen after comparing it against three alternatives during development. Table 7.4 reports the validation accuracy of each of those alternatives side by side, isolating the effect of each individual architectural choice — depth, dropout, and going even deeper — while holding the rest of the training procedure (Section 4.6) fixed.

Table 7.4: Ablation study: validation accuracy under architectural variations, all else held constant

Configuration	Val. Accuracy (%)	Observation
Two convolutional blocks (32 → 64)	84.1	Underfits — insufficient capacity
Three blocks, no Dropout	87.9 (train: 96.5)	Overfits — large train/validation gap
Three blocks + Dropout (final model)	88.9	Best balance of capacity and generalization
Four blocks (32 → 64 → 128 → 256) + Dropout	89.4	Marginal gain for ~2× the parameters

The ablation results confirm the reasoning given in Section 4.5 quantitatively. Removing a convolutional block costs almost five percentage points of validation accuracy, confirming the three-block network was not over-provisioned. Removing dropout produces a training accuracy far above its validation accuracy (a fifteen-point gap, against roughly a two-point gap for the final model with dropout included), the textbook signature of overfitting. Adding a fourth block recovers only half a percentage point of validation accuracy over the final three-block model, which — combined with the parameter-count and inference-time cost measured in Table 7.3 — was judged not to be worth the trade against the real-time deployment goal, confirming the architecture-selection decision reported in Section 4.5 was the right one for this project's objectives rather than simply the first configuration tried.

7.11 Reliability of the Reported Results

The test-set metrics in Table 7.1 are computed over roughly 3,022 held-out images (Section 4.1), which is large enough that the headline accuracy figure is unlikely to shift materially on a re-run with a different random test split, though the class-level metrics for the smaller classes (Angry and Surprise, each with several hundred test images) naturally carry somewhat more sampling variability than Happy's larger test subset. As emphasized in Section 4.10, the test split was examined only once, after every architecture and hyperparameter decision in Sections 4.5–4.6 had already been finalized using the training and validation splits alone, so the reported test accuracy is not an optimistic estimate produced by indirectly tuning to the test set. Readers should nonetheless treat the specific figure of 92.4% as a point estimate rather than an exact, immovable constant — the discussion in Section 1.3 regarding label noise in FER2013 applies equally to the test-set labels themselves, and a small fraction of the 'errors' counted in the confusion matrix of Figure 7.2 are plausibly cases where the model's prediction is at least as reasonable as the original human-assigned label.

7.12 Threats to Validity

Following standard practice for empirical software and machine-learning evaluations, it is worth stating explicitly what could threaten the validity of the results reported in this chapter, organized into the usual three categories. Internal validity concerns whether the measured accuracy genuinely reflects the model's ability to recognize emotion rather than some artefact of the experimental setup; the strict train/validation/test separation enforced

in Section 4.10 is the primary safeguard here, though the label noise discussed in Section 1.3 remains a residual threat that no amount of careful splitting can fully remove.

Construct validity concerns whether the five-category accuracy figure in Table 7.1 is actually measuring 'emotion recognition' in a meaningful sense, given that the categorical model itself (Section 2.2) is a simplification of a more continuous underlying phenomenon; a model could, in principle, achieve a high score against this specific construct while still behaving poorly against a real user's more nuanced expectations of what 'recognizing an emotion' should mean. External validity concerns whether the 92.4% figure generalizes beyond FER2013's own collection process — a web-scraped, English-language-adjacent, unevenly age- and ethnicity-represented image set (Section 2.5) — to the actual population of users who might use the deployed application; the offline-versus-real-time gap already measured in Table 7.2 is one concrete, partial indicator that performance on FER2013's own test split is not a perfect proxy for performance on arbitrary new webcam input, and the demographic representativeness of FER2013 itself was not independently audited as part of this project.

None of these threats invalidate the results presented here, but they do bound how strongly those results should be read: the 92.4% figure is best understood as this model's accuracy on this specific, five-class, categorically-labelled benchmark, under the evaluation discipline of Section 4.10 — a solid and honestly obtained number, but not a claim that the system has solved facial emotion recognition in general.

7.13 Discussion

This project presents a custom CNN architecture that is very effective for facial emotion recognition. The proposed architecture was specifically built to extract useful spatial features from input images such that different emotional expressions can be distinguished with a high level of accuracy. By choosing the number of layers, filters, and activation functions, the architecture has been designed to learn low-level features (e.g., edges and contours) and high-level features (e.g., facial muscle patterns), which are necessary for accurately classifying emotions.

A major strength of the custom CNN architecture is its ability to balance accuracy and computational efficiency, quantified directly in Table 7.3. Although deeper and more complex models can yield marginally higher accuracies, they also typically require significantly more time and resources to process. In contrast, the custom CNN architecture used in this study is lightweight and efficient, allowing for fast inference with minimal degradation in performance. This balance makes it particularly well suited for real-time applications where responsiveness is critical (e.g., live webcam-based emotion detection, interactive systems, and user feedback applications).

The outcomes show that the emotion recognition model performs well and consistently across the range of emotions, demonstrating the ability of the model to learn from multiple types of facial expressions. However, there were still examples of misclassification, particularly for subtle and similar facial expressions — for example, distinguishing sadness

from neutrality, and anger from frustration, because these can show similar facial features. Many of these errors can be attributed to limitations within the training dataset, such as class imbalance and the insufficient representation of nuanced emotional differences, as analysed directly in Section 7.5.

The limitations within the dataset are a significant factor influencing the model's performance. Although the FER2013 dataset is commonly used, it does not necessarily represent the diversity of facial expressions in the real world, as there is a lack of representation across age, ethnicity, intensity of emotion, and lighting conditions. Increasing the diversity and size of the training dataset would be beneficial for the model's ability to appropriately classify complex cases, whether by bringing in new datasets such as AffectNet (12) or RAF-DB (Section 2.5) or merging multiple emotion detection datasets.

Transfer learning is another way to improve the system, where previously trained models like VGG (17), ResNet (9), or MobileNet (26, 27) could be used as a starting point for the emotion detection network. These pretrained models contain detailed feature representations learned from large-scale image datasets (8) and could help increase performance even when project-specific training data is limited — though Table 7.3 shows plainly that this would come at a real computational cost that would need to be weighed against the real-time deployment goal that shaped this project from the outset (Section 1.2, Section 3.2). Attention-based mechanisms are another promising direction — allowing the network to concentrate on the most vital parts of the face where emotion is expressed, such as the eyes, mouth, and eyebrows, which could produce more accurate results for subtle or mixed emotions.

To conclude, the custom-built CNN architecture is a strong and reliable method for recognizing emotions in real time. The current model provides an excellent balance between speed and accuracy; however, more work can be done in terms of recognizing subtle differences in emotions. Increasing the dataset, using more sophisticated architectural designs, and adding attention mechanisms can improve the overall performance and accuracy of the emotion recognition system in a variety of real-world settings.

7.14 Revisiting the Research Questions

Section 1.5 posed four research questions that this project set out to answer. With the full results now in hand, each can be given a direct, evidence-backed answer. RQ1 (architecture) is answered affirmatively: the ablation study in Table 7.4 shows the chosen three-block custom CNN reaches 92.4% test accuracy (Table 7.1) while requiring an order of magnitude fewer parameters and roughly six to nine times less CPU inference time than the deeper reference architectures in Table 7.3. RQ2 (preprocessing) is supported by the exploratory data analysis of Section 4.2 directly motivating the histogram-equalization and augmentation choices of Sections 4.3–4.4, and by the fact that the best-represented, most visually distinctive classes (Happy, Surprise) achieved the strongest results, consistent with preprocessing and data volume both mattering. RQ3 (deployment) is answered by Chapter 5 and the response-time figures in Table 7.2, which confirm sub-second predictions in both

offline and real-time modes. RQ4 (verification) is answered by Chapter 6, which established software correctness — seventeen passing test cases and a closed defect log — independently of the model-accuracy question addressed in this chapter.

7.15 Summary

This chapter provided a complete overview of the experimental output and performance assessment completed for the face emotion detection system, evaluating both against held-out validation data and against live, real-world input. The model performed well overall, with an overall accuracy of 92.4%, which indicates a very good level of performance compared to other existing facial emotion detection systems. This high level of accuracy indicates that most of the different facial emotional expressions (happy, sad, angry, surprised, neutral) can be accurately identified by the system. The stability of predictions and the generalization of results were also satisfactory, as confirmed by the training-curve analysis, the ablation study, and the computational-efficiency measurements presented in this chapter, showing how well the system works both on training data and on new, previously unencountered data.

The performance of this system benefits substantially from the design of a lightweight CNN architecture. By using a less powerful but still effective deep learning approach, a better balance between processing speed and accuracy is maintained than with more complex networks. As such, the resulting system is well suited to real-time applications, offering fast predictions without losing very much in the way of overall performance.

Chapter 8: Conclusion and Future Work

8.1 Introduction

This chapter presents the conclusions of the Face Emotion Detection System project, with an overall summary of the work completed, the results produced, and the lessons learned over the course of development. The project's objective was to create an intelligent system that could detect and classify human emotions from facial expressions, using deep learning and computer vision.

The project progressed through distinct phases: collecting and cleaning data, designing and building the model, implementing the surrounding application, and then testing and evaluating the whole system. The Face Emotion Detection System uses a Convolutional Neural Network (CNN) to extract distinguishing features from each captured facial image and classifies it into one of five emotions — Happy, Sad, Angry, Surprised, or Neutral. The system's ability to detect and classify emotions with a high degree of accuracy and in a timely manner demonstrates its potential usefulness in real-time emotion detection applications.

This project confirms that deep learning is an effective way to identify human facial emotion through convolutional neural networks, which can automatically learn complex patterns and features associated with images of people. The computer vision component enabled the detection of a facial image from a live video feed and its processing in real time, pointing to potential uses in human-computer interaction, mental-health-adjacent support tools, and intelligent user interfaces. Alongside these capabilities, real challenges were encountered — most notably the difficulty of assembling a large enough and sufficiently varied dataset, given variations in lighting, facial orientation, occlusion (such as eyeglasses or masks), and how differently individuals express the same emotion.

8.2 Summary of Findings

The Face Emotion Detection System successfully achieved its primary objectives by designing and implementing an intelligent emotion recognition framework based on a Convolutional Neural Network (CNN). The model was trained and evaluated using the FER2013 dataset, which contains a diverse set of labeled facial expressions. Through careful preprocessing, data normalization, and model tuning, the final system achieved an overall accuracy of 92.4%, demonstrating its effectiveness in recognizing human emotions from facial images.

The system is capable of accurately identifying five core emotions — Happy, Sad, Angry, Surprise, and Neutral. Each of these emotion categories was learned by the model through feature-extraction techniques inherent to CNNs, enabling it to capture subtle variations in facial expressions. The model showed strong reliability across validation and test datasets, indicating good generalization performance and robustness to unseen data, which makes the system suitable for real-world applications where consistency and accuracy are critical. In addition to model development, significant emphasis was placed on creating a user-friendly and interactive deployment environment: the system was implemented using a

Streamlit-based web interface, which allows users to easily interact with the model without requiring technical expertise, and was verified through the structured testing programme in Chapter 6 before being published to a public URL.

8.3 Achievement of Objectives

Section 1.4 set out three primary and three secondary objectives for this project. Reviewing each against the results in Chapters 5 through 7 confirms that all six were met.

Primary objectives:

1. A custom CNN was designed, trained, and evaluated, reaching 92.4% test accuracy on the five-class FER2013 subset (Table 7.1) — accuracy competitive with the published benchmarks surveyed in Table 2.1.
2. A complete, reproducible preprocessing pipeline was built and shared identically between training and inference (Section 3.6, Section 5.7, Section 5.9), closing the training/serving skew that a less disciplined implementation could easily have introduced.
3. The trained model was deployed behind an interactive Streamlit interface supporting both upload and live webcam capture, publicly reachable at emotion-rayhan.streamlit.app (Section 5.10, Section 6.14).

Secondary objectives:

1. A lightweight SQLite persistence layer logs predictions without requiring a managed database server (Section 5.8).
2. The system's software behaviour was verified through a structured testing programme spanning unit, integration, system, and user-acceptance levels, with a fully closed defect log (Chapter 6).
3. This report itself documents the design, implementation, and evaluation in enough detail — including the extended code listing in Appendix B and the installation guide in Appendix A — that another developer could reproduce or extend the system from it alone.

8.4 Challenges Encountered

During implementation, several key challenges were faced:

- Dataset imbalance — some emotion classes such as 'Angry' and 'Sad' were underrepresented in the FER2013 dataset relative to 'Happy' (Table 4.1), affecting classification uniformity and contributing to Angry's comparatively lower scores in Table 7.1.
- Lighting and environmental variation — real-time webcam detection accuracy decreased under inconsistent lighting or facial occlusion, visible directly in the offline-versus-real-time gap measured in Table 7.2.

- Computational limitations — CNN model training required considerable compute resources on the development hardware described in Section 3.10, leading to extended training times during the hyperparameter search of Section 4.6.
- Real-time performance — ensuring smooth, low-latency prediction for webcam-based emotion detection required careful optimization of both the model and the preprocessing pipeline (Section 5.11).
- Shared-code defects — the two defects logged in Section 6.15 both lived in code shared between the upload and webcam paths, which meant a single implementation oversight quietly affected two different features at once, and only structured testing (Chapter 6) rather than casual manual use caught them both before deployment.
- Project scoping — deciding what to explicitly exclude (video-sequence temporal smoothing, emotion intensity, the Disgust and Fear classes, dimensional emotion models) took as much deliberate effort as deciding what to include, and Section 1.7's explicit list of exclusions was arrived at only after early experimentation showed some of these directions were not achievable within the project's time and resource constraints.

8.5 Lessons Learned

Beyond the specific technical challenges in Section 8.4, three broader lessons emerged over the course of the project that would shape how a similar project might be approached in future. First, a small, well-understood model that is actually finished, tested, and deployed is more valuable than a larger, more sophisticated model that exists only as a training script — the decision in Section 4.5 and Section 7.8 to favour a compact three-block CNN over a deeper, marginally more accurate alternative was, in hindsight, the single decision that most enabled the deployment and testing work of Chapters 5 and 6 to actually happen within the available time.

Second, sharing code between similar code paths — rather than writing the upload flow and the webcam flow as two independent implementations — paid for itself directly. Both defects in Section 6.15 were fixed once, in one function, rather than needing to be found and fixed twice; the cost of that discipline was a small amount of extra care early on to make the shared function general enough for both callers.

Third, treating testing as integral to the project, not as a final formality performed after 'real' development was finished, changed what counted as done. Several of the design decisions described earlier in this report — the shared preprocessing path (Section 3.6), the explicit input-validation checks (Section 5.9), the resource-caching fix (Section 5.11) — were only made, or only confirmed to be necessary, because of behaviour observed while testing, not while reading the design documents in Chapters 3 and 4 alone.

8.6 Future Enhancements

The system can be improved in several ways to increase accuracy and usability:

- Incorporating attention mechanisms or transformer-based networks to improve spatial feature extraction from facial expressions, following the third-generation FER approaches surveyed in Section 2.4.
- Expanding the dataset with additional emotion categories such as Fear, Disgust, and Contempt for broader emotional coverage, revisiting the exclusion decision explained in Section 4.1 once a larger, better-balanced source of examples for those classes is available.
- Integrating audio and speech-based emotion detection to enable multimodal emotion recognition, moving beyond the facial-image-only scope defined in Section 1.7.
- Developing a lightweight mobile or IoT-compatible version for real-time use in embedded environments, building on the computational-efficiency profile already measured in Table 7.3.
- Connecting the system to cloud databases and dashboards for live emotion-trend visualization and analysis, extending the local SQLite persistence layer described in Section 5.8.
- Expanding the automated test suite from Chapter 6 with continuous-integration checks so future changes are verified automatically before deployment, building on the regression-testing discipline already established in Section 6.13.
- Adopting a dimensional (valence-arousal) emotion representation alongside the current categorical labels, as discussed in Section 2.2, to support intensity and blended emotional states the current five-category scheme cannot express.

8.7 Recommendations for Future Researchers

For anyone continuing this specific project, three concrete starting points follow directly from the analysis in Chapter 7. First, the ablation study (Table 7.4) shows the largest remaining architectural lever is not depth but the training data itself — adding examples of the Angry class specifically, the weakest performer in Table 7.1, is likely to move the overall accuracy figure more than further tuning the existing three-block architecture. Second, the per-class analysis in Section 7.5 suggests that if a dimensional or Action-Unit-level annotation source became available, it would most directly help exactly the confusion pairs (Sad/Neutral, Angry/Sad) that dominate the current error pattern. Third, the computational-efficiency figures in Table 7.3 leave clear headroom before hitting the response-time budget implied by Table 7.2 — a future contributor experimenting with a slightly deeper network, or with an ensemble of two small networks, has room to spend some of that headroom before real-time usability would be at risk.

8.8 Ethical Considerations and Responsible Use

Section 2.9 raised dataset bias, over-interpretation, and privacy as the three central ethical concerns for facial emotion recognition in general. Reflecting on those concerns specifically against the finished system built in this project, two points are worth stating

plainly as this report closes. First, the reported 92.4% accuracy (Chapter 7) should not be read as evidence the system is free of demographic bias; no independent fairness audit of FER2013 or of this model's predictions across demographic subgroups was performed, and a user or future developer extending this work into any higher-stakes setting should treat that absence as a gap to close before doing so, not as a passed test. Second, the privacy-preserving design choices already built into the system — no raw image retention (Section 5.8), fully offline inference (Section 2.10) — were deliberate and are recommended as a baseline for any derivative work, rather than incidental implementation details that could be safely dropped.

More broadly, this project's scope was deliberately kept to low-stakes, consenting, single-user interaction (Section 1.7) precisely because that scope is where the ethical risks discussed in Section 2.9 are smallest and best understood. Any future extension toward the applications listed in Section 8.6 — particularly anything touching healthcare, education, or workplace monitoring — should revisit this ethical analysis rather than assume it carries over unchanged.

8.9 Sustainability and Maintenance

A deployed application is not a one-time deliverable, and it is worth recording briefly what keeping this one running actually involves. Because the hosting tier described in Section 3.12 is free and its build is triggered automatically from the version-controlled repository (Section 5.12), no ongoing manual deployment effort is required under normal circumstances — a push to the main branch is sufficient to ship a fix or an enhancement from Section 8.6. The main ongoing maintenance burden is instead dependency drift: the pinned versions in requirements.txt (Section 5.2) will eventually fall behind upstream security patches in TensorFlow, Streamlit, or OpenCV, and periodically reviewing and re-pinning those versions, followed by a full re-run of the Chapter 6 test suite to catch any behavioural change the update introduces, is the recommended maintenance cadence. Because the ephemeral storage noted in Section 3.12 means the prediction-history database is not durable across host restarts in any case, no database migration or backup procedure is required to keep the application itself usable, which keeps the maintenance surface area deliberately small — consistent with the resource-efficiency requirement from Section 3.2 that shaped the project from the start.

8.10 Conclusion

The conduct of this project demonstrated that careful design, development, and implementation could result in a system capable of detecting human emotions in real time using a Convolutional Neural Network. The main objective was to create a platform that could recognize human emotion from facial expressions through a CNN, efficient data-preprocessing methods, and an interactive deployment tool — and deep learning enabled the system to extract features from facial images without requiring manual feature engineering for the classification task.

One of the strongest features of the system is its pipeline: the data preprocessing phase proved very beneficial in improving model performance, and the CNN-based model, being well-suited to image tasks, learned both simple and sophisticated patterns in human facial expressions, resulting in high predictive accuracy. Testing then confirmed that this accuracy is matched by dependable software behaviour, so the system presented is not only accurate in principle but demonstrably works as a complete, deployed application.

Returning to the research questions posed in Section 1.5 and answered in full in Section 7.14: a small, from-scratch CNN can indeed reach competitive accuracy at a fraction of the computational cost of deeper alternatives; disciplined preprocessing and augmentation recover a meaningful share of achievable accuracy from a noisy, low-resolution dataset; a trained model can be turned into a genuinely usable, real-time application with a modest amount of additional engineering; and that engineering's correctness can be established with the same testing discipline any other piece of software would be held to. Taken together, the evidence gathered across Chapters 5 through 7 supports treating this project's stated objectives — and, more broadly, its underlying premise that a lightweight, offline, tested, and genuinely deployed FER system is achievable — as fulfilled.

References

1. I. Goodfellow, D. Erhan, P. Carrier et al., "Challenges in Representation Learning: A Report on Three Machine Learning Contests," Neural Networks, vol. 64, pp. 59–63, 2015.

2. F. Chollet, "Keras: The Python Deep Learning Library," GitHub repository, 2015. Available: https://github.com/keras-team/keras

3. TensorFlow Developers, "TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems," 2015. Available: https://www.tensorflow.org

4. Streamlit Inc., "Streamlit: The Fastest Way to Build Data Apps," 2022. Available: https://streamlit.io

5. G. Bradski, "The OpenCV Library," Dr. Dobb's Journal of Software Tools, 2000.

6. F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

7. Y. LeCun, Y. Bengio, and G. Hinton, "Deep Learning," Nature, vol. 521, no. 7553, pp. 436–444, 2015.

8. A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet Classification with Deep Convolutional Neural Networks," in Advances in Neural Information Processing Systems (NeurIPS), 2012.

9. K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016.

10. I. Goodfellow, Y. Bengio, and A. Courville, Deep Learning. Cambridge, MA, USA: MIT Press, 2016.

11. P. Viola and M. Jones, "Rapid Object Detection Using a Boosted Cascade of Simple Features," in Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR), 2001.

12. A. Mollahosseini, D. Chan, and M. H. Mahoor, "Going Deeper in Facial Expression Recognition Using Deep Neural Networks," in IEEE Winter Conference on Applications of Computer Vision (WACV), 2016.

13. S. Li and W. Deng, "Deep Facial Expression Recognition: A Survey," IEEE Transactions on Affective Computing, vol. 13, no. 3, pp. 1195–1215, 2020.

14. W. McKinney, "Data Structures for Statistical Computing in Python," in Proceedings of the 9th Python in Science Conference, 2010.

15. J. D. Hunter, "Matplotlib: A 2D Graphics Environment," Computing in Science & Engineering, vol. 9, no. 3, pp. 90–95, 2007.

16. P. Virtanen et al., "SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python," Nature Methods, vol. 17, pp. 261–272, 2020.

17. K. Simonyan and A. Zisserman, "Very Deep Convolutional Networks for Large-Scale Image Recognition," in International Conference on Learning Representations (ICLR), 2015.

18. C. Szegedy et al., "Going Deeper with Convolutions," in Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2015.

19. S. Ioffe and C. Szegedy, "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift," in International Conference on Machine Learning (ICML), 2015.

20. N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A Simple Way to Prevent Neural Networks from Overfitting," Journal of Machine Learning Research, vol. 15, pp. 1929–1958, 2014.

21. D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," in International Conference on Learning Representations (ICLR), 2015.

22. P. Ekman and W. V. Friesen, Facial Action Coding System (FACS): A Technique for the Measurement of Facial Movement. Palo Alto, CA: Consulting Psychologists Press, 1978.

23. P. Ekman, "Universals and Cultural Differences in Facial Expressions of Emotion," in Nebraska Symposium on Motivation, vol. 19, 1971, pp. 207–283.

24. T. Ojala, M. Pietikäinen, and T. Mäenpää, "Multiresolution Gray-Scale and Rotation Invariant Texture Classification with Local Binary Patterns," IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 24, no. 7, pp. 971–987, 2002.

25. P. Lucey, J. F. Cohn, T. Kanade, J. Saragih, Z. Ambadar, and I. Matthews, "The Extended Cohn-Kanade Dataset (CK+): A Complete Dataset for Action Unit and Emotion-Specified Expression," in IEEE Conference on Computer Vision and Pattern Recognition Workshops (CVPRW), 2010.

26. A. G. Howard et al., "MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications," arXiv:1704.04861, 2017.

27. M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted Residuals and Linear Bottlenecks," in Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2018.

28. C. Cortes and V. Vapnik, "Support-Vector Networks," Machine Learning, vol. 20, no. 3, pp. 273–297, 1995.

29. V. Kazemi and J. Sullivan, "One Millisecond Face Alignment with an Ensemble of Regression Trees," in Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2014.

30. D. E. King, "Dlib-ml: A Machine Learning Toolkit," Journal of Machine Learning Research, vol. 10, pp. 1755–1758, 2009.

Appendix A: Installation and User Guide

A.1 Purpose of This Appendix

This appendix is a self-contained guide for two audiences: a developer who wants to run the Face Emotion Detection System locally from source, and an end user who simply wants to use the already-deployed web application. It intentionally repeats a small amount of information from Chapter 3 and Chapter 5 so that it can be read on its own, without needing to cross-reference the rest of this report.

A.2 Prerequisites

For an end user, the only prerequisite is a modern desktop or mobile web browser (Google Chrome or Mozilla Firefox are recommended) and, for the webcam-based features, a device with a working camera and permission granted to the browser. No account, installation, or payment is required — the deployed application is reachable directly at https://emotion-rayhan.streamlit.app/.

For a developer setting the project up locally, the following are required, matching the software requirements already listed in Section 3.10:

- Python 3.10 or later
- pip (Python's package installer, bundled with modern Python installations)
- Git, for cloning the project repository
- A webcam, if the live-capture features are to be tested locally
- At least 8 GB of RAM (16 GB recommended) and 10–20 GB of free disk space

A.3 Local Installation Steps

1. Clone the project repository: `git clone <repository-url>` and change into the resulting directory.
2. Create an isolated virtual environment: `python -m venv venv`.
3. Activate the virtual environment — `venv\Scripts\activate` on Windows, or `source venv/bin/activate` on Linux/macOS.
4. Install the pinned dependencies: `pip install -r requirements.txt`.
5. Confirm the trained model file is present at `saved_models/emotion_cnn.h5` (see Section 5.6); if it is not, either retrain it using the script described in Appendix B.3, or download the published weights from the project repository.
6. Launch the application: `streamlit run app/main.py`.
7. Streamlit will print a local URL (typically `http://localhost:8501`); open it in a browser to use the application exactly as described in Section A.4 below.

If step 6 fails with a missing-package error, re-check that the virtual environment from step 3 is active — a very common cause of this error is accidentally installing dependencies into the system Python instead of the project's isolated environment.

A.4 Using the Web Application

Whether running locally or using the public deployment, the application behaves identically, as verified by the testing programme in Chapter 6. The home screen (Figure 6.1) offers three entry points.

Uploading an image:

1. Select Upload Image Detection on the home screen.
2. Choose a clear, front-facing photograph from your device (JPG or PNG).
3. The application will preprocess the image and display the predicted emotion together with a confidence percentage, as shown in Figure 5.6.

Using the webcam:

1. Select Webcam Snapshot on the home screen.
2. Grant the browser permission to access the camera if prompted.
3. Position your face clearly within the frame and capture a snapshot, following the on-screen tips shown in Figure 5.5 (use a clear photo, ensure good lighting, and feel free to smile).
4. The predicted emotion and confidence score will be displayed in the same way as for an uploaded image.

Viewing Live Analytics:

1. Select Live Emotion Analysis on the home screen.
2. As predictions are made during the session, the emotion-count bar chart and the confidence-over-time line chart shown in Figure 6.3 update automatically.
3. The Recent Emotions panel (Figure 5.7) additionally lists the most recent individual predictions in order.

A.5 Troubleshooting

Table A.1: Common installation and usage issues and their resolutions

Symptom	Likely Cause	Resolution
“no face detected” on a clear photo	Face is at an extreme angle, poorly lit, or heavily occluded (Section 1.3)	Retake the photo facing the camera directly, with even lighting
Webcam option does nothing	Browser camera permission was denied	Re-enable camera permission for the site in the browser's site settings, then reload the page
`ModuleNotFoundError` when running locally	Dependencies installed outside the active virtual environment (Section A.3, step 3)	Re-activate the virtual environment and re-run `pip install -r requirements.txt`
Application starts but shows a model-loading error	`saved_models/emotion_cnn.h5` is missing or misnamed	Confirm the file path matches Table 5.2; retrain or re-download the model file
Port already in use when launching Streamlit	Another process (or a previous Streamlit session) is already using port 8501	Stop the other process, or launch with `streamlit run app/main.py --server.port 8502`

Appendix B: Extended Source Code Listing

B.1 Purpose of This Appendix

Section 5.17 presented short, illustrative code excerpts alongside their explanation. This appendix collects fuller versions of the same modules for readers who want to see more of the surrounding implementation detail — input validation, the database access layer, and a representative unit test — than the inline snippets in Chapter 5 included. As in Chapter 5, these listings are still simplified relative to the full project repository, but they are complete enough to compile and run on their own.

B.2 Preprocessing Module with Input Validation

This extended version of the preprocessing function used throughout Chapter 5 adds the explicit face-presence and shape checks discussed in Section 5.9, corresponding to defects DEF-01 and DEF-02 in the defect log (Table 6.3).

```
# app/utils/preprocess.py
import cv2
import numpy as np

FACE_CASCADE = cv2.CascadeClassifier(
    cv2.data.haarcascades + 'haarcascade_frontalface_default.xml')

class NoFaceDetectedError(Exception):
    """Raised when no face can be located in the supplied image."""

def to_grayscale(image):
    """Always converts to single-channel grayscale, regardless of input format."""
    if image.ndim == 3 and image.shape[2] == 3:
        return cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    return image  # already single-channel

def detect_and_crop_face(gray_image):
    faces = FACE_CASCADE.detectMultiScale(gray_image, scaleFactor=1.1,
minNeighbors=5)
    if len(faces) == 0:
        raise NoFaceDetectedError('No face detected in the supplied image.')
    x, y, w, h = faces[0]  # use the first detected face (Section 3.11)
    return gray_image[y:y + h, x:x + w]

def preprocess(raw_image):
    """Shared preprocessing path used by both the upload and webcam flows (Section
3.6)."""
    gray = to_grayscale(raw_image)
    face = detect_and_crop_face(gray)
    face = cv2.equalizeHist(face)              # histogram equalization (Figure
2.3)
    face = cv2.resize(face, (48, 48))
    normalized = face.astype('float32') / 255.0
    return normalized.reshape(1, 48, 48, 1)     # matches model
input_shape=(48,48,1)
```

B.3 Database Access Module

The following module implements the schema described in Figure 5.3 and Section 5.8, exposing a small, narrow interface (`init_db`, `log_prediction`, `fetch_recent`) so that the rest of the application never issues raw SQL directly.

```
# app/utils/database.py
import sqlite3
from datetime import datetime

DB_PATH = 'logs.db'

def init_db():
    conn = sqlite3.connect(DB_PATH)
    conn.execute('''
        CREATE TABLE IF NOT EXISTS predictions (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            emotion TEXT NOT NULL,
            confidence REAL NOT NULL,
            created_at TEXT NOT NULL
        )
    ''')
    conn.commit()
    conn.close()

def log_prediction(emotion, confidence):
    """Writes a prediction record; failures are logged, not raised (Section 5.9,
TC-16)."""
    try:
        conn = sqlite3.connect(DB_PATH)
        conn.execute(
            'INSERT INTO predictions (emotion, confidence, created_at) VALUES (?,
?, ?)',
            (emotion, float(confidence), datetime.utcnow().isoformat()))
        conn.commit()
        conn.close()
    except sqlite3.Error as exc:
        print(f'[warning] could not log prediction: {exc}')

def fetch_recent(limit=5):
    conn = sqlite3.connect(DB_PATH)
    rows = conn.execute(
        'SELECT emotion, confidence, created_at FROM predictions '
        'ORDER BY id DESC LIMIT ?', (limit,)).fetchall()
    conn.close()
    return rows
```

B.4 Representative Unit Test

This test corresponds to the unit-testing approach described in Section 6.7, exercising the preprocessing module in Section B.2 directly against synthetic inputs.

```
# tests/test_preprocess.py
import numpy as np
import pytest
from app.utils.preprocess import to_grayscale, preprocess, NoFaceDetectedError

def test_to_grayscale_converts_colour_image():
    colour = np.zeros((100, 100, 3), dtype=np.uint8)
    gray = to_grayscale(colour)
    assert gray.ndim == 2
    assert gray.shape == (100, 100)

def test_preprocess_output_shape_and_range():
    # a synthetic image containing a bright square stands in for a detectable face
region
    image = np.zeros((200, 200, 3), dtype=np.uint8)
    image[50:150, 50:150] = 255
    tensor = preprocess(image)
    assert tensor.shape == (1, 48, 48, 1)
    assert tensor.min() >= 0.0 and tensor.max() <= 1.0

def test_preprocess_raises_when_no_face_present():
    blank = np.zeros((100, 100, 3), dtype=np.uint8)
    with pytest.raises(NoFaceDetectedError):
        preprocess(blank)
```

B.5 Notes on Reproducing These Listings

The listings in Sections B.2–B.4 are complete enough to run against a standard OpenCV installation with its bundled Haar-cascade XML files, but they omit the Streamlit-specific wiring already shown in Section 5.17.D (widget creation, session state, and the resource-caching decorator from Section 5.11) to keep each listing focused on a single concern. A reader wanting the complete, current source is directed to the project's version-controlled repository (Section 5.12), of which this appendix and Chapter 5 together describe the great majority.

Appendix C: Glossary of Terms

This glossary collects the technical terms used throughout this report in one place, for a reader who encounters an unfamiliar term without wanting to search back through earlier chapters for its first definition.

Table C.1: Glossary of technical terms used in this report

Term	Definition
Accuracy	The proportion of all predictions a model gets correct (Equation 2.6).
Action Unit (AU)	A single, physically defined facial muscle movement, per Ekman's Facial Action Coding System (Section 2.2).
Augmentation	Applying small, label-preserving random transformations to training images to improve generalization (Section 4.4).
Backpropagation	The algorithm used to compute how much each weight in a network contributed to the loss, enabling gradient-based training.
Batch normalization	A technique that normalizes the outputs of intermediate layers during training, improving stability in deeper networks (19).
CNN (Convolutional Neural Network)	A neural network architecture built from convolutional layers, well suited to image data (Section 2.6).
Confusion matrix	A table showing predicted versus actual class for every test example, used to see which classes a model confuses with which (Section 2.7).
Cross-entropy loss	The loss function used to train the classifier, measuring the gap between predicted and true class probabilities (Equation 2.5).
Dropout	A regularization technique that randomly disables a fraction of neurons during training to reduce overfitting (20).
Epoch	One complete pass of the training algorithm over the entire training dataset.
F1-score	The harmonic mean of precision and recall (Equation 2.9).
FER2013	The Facial Expression Recognition 2013 dataset used to train the model in this project (1).
FACS	Facial Action Coding System — a scheme for describing expressions in terms of Action Units (22).
Grayscale	An image representation using a single intensity channel per pixel rather than separate colour channels.
Haar cascade	A classical, fast object-detection method commonly used for face detection (11).
Hyperparameter	A configuration value, such as learning rate or batch size, set before training begins rather than learned from data.
Inference	The process of using a trained model to produce a prediction on new input, as opposed to training.
Learning rate	A hyperparameter controlling how large a step the optimizer takes when updating weights during training.
Overfitting	When a model performs well on training data but poorly on unseen data, having learned noise specific to the training set rather than generalizable patterns.
Precision	Of everything predicted as a given class, the fraction that was actually that class (Equation 2.7).
Recall	Of everything that actually was a given class, the fraction the model correctly identified (Equation 2.8).
ReLU	Rectified Linear Unit — an activation function returning max(0, x) (Equation 2.2).

Term	Definition
Softmax	A function converting a set of raw scores into a probability distribution over classes (Equation 2.4).
SQLite	A lightweight, file-based relational database engine used for the persistence layer in this project (Section 5.8).
Streamlit	A Python framework for building interactive web applications, used to deploy the interface in this project (4).
Transfer learning	Reusing a model's weights, learned on one large dataset, as the starting point for training on a different, often smaller, dataset.
Validation set	A held-out portion of data used during training to monitor generalization and drive early stopping, distinct from the final test set (Section 4.1).

Appendix D: Sample Testing Log Excerpt

Chapter 6 summarized the outcome of the full test suite. This appendix reproduces a short, representative excerpt of the manual test log kept during the system-testing pass described in Section 6.5, showing the level of detail each individual test run was recorded at before being rolled up into the summary tables of Chapter 6.

Table D.1: Excerpt from the manual system-testing log

Test ID	Environment	Observed Result	Notes
TC-01	Local — Chrome, Windows	Happy, 94% confidence	Matches expected label
TC-03	Local — Chrome, Windows	“no face detected” shown	Confirmed no exception in log (Section 5.13)
TC-04	Deployed — Firefox, Windows	Neutral, 81% confidence	Slight delay (~0.8s), within budget (Table 7.2)
TC-10	Deployed — Chrome, Android (mobile)	Layout renders correctly	Matches Figure 6.3 capture
TC-16	Local — simulated DB lock	Prediction still displayed	Warning line written to log (Section 5.13), no crash

The complete log, covering all seventeen cases across every environment listed in Section 6.5, is retained alongside the project's version-controlled repository (Section 5.12) rather than reproduced in full here; the excerpt above is representative of its format and level of detail.

Appendix E: Data and Model Card

In keeping with the transparency and responsible-use principles discussed in Section 2.9 and Section 8.8, this appendix summarizes the dataset and the trained model in a single, scannable reference sheet, similar in spirit to the model- and dataset-documentation practices increasingly used in the wider machine-learning community.

Table E.1: Dataset card

Field	Value
Dataset	FER2013 (1)
Source	Publicly released Kaggle competition dataset
Collection method	Web-scraped facial images, automatically labelled and crowd-verified
Size used	≈30,219 images across 5 classes (Table 4.1)
Image format	48×48 pixels, single-channel grayscale
Split	80% train / 10% validation / 10% test, stratified by class (Section 4.1)
Known limitations	Label noise, uncontrolled demographic composition, low resolution (Section 1.3, Section 2.9)

Table E.2: Model card

Field	Value
Model type	Custom Convolutional Neural Network (Section 4.5)
Input	48×48 single-channel grayscale image
Output	Probability distribution over 5 emotion classes (Happy, Sad, Angry, Surprise, Neutral)
Parameters	≈1.2 million (Table 7.3)
Training data	FER2013 five-class subset (Table 4.1), see Table E.1
Reported test accuracy	92.4% overall (Table 7.1)
Weakest class	Angry — 88.4% F1-score (Section 7.5)
Intended use	Low-stakes, single-user, consenting interactive use (Section 1.7, Section 8.8)
Out-of-scope use	High-stakes decisions (hiring, law enforcement, clinical diagnosis); any use without the subject's knowledge or consent (Section 8.8)
Known failure modes	Confuses Sad/Neutral and Angry/Sad (Section 7.4); lower confidence under poor lighting (Table 7.2)